МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена трудового Красного Знамени федеральное государственное бюджетное

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Программная инженерия

Отчет по лабораторной работе № 4

Разработка RESTful API: спецификация и архитектура

Выполнил: студент группы

БПИ2503

Яричевский Даниил

Москва

2025

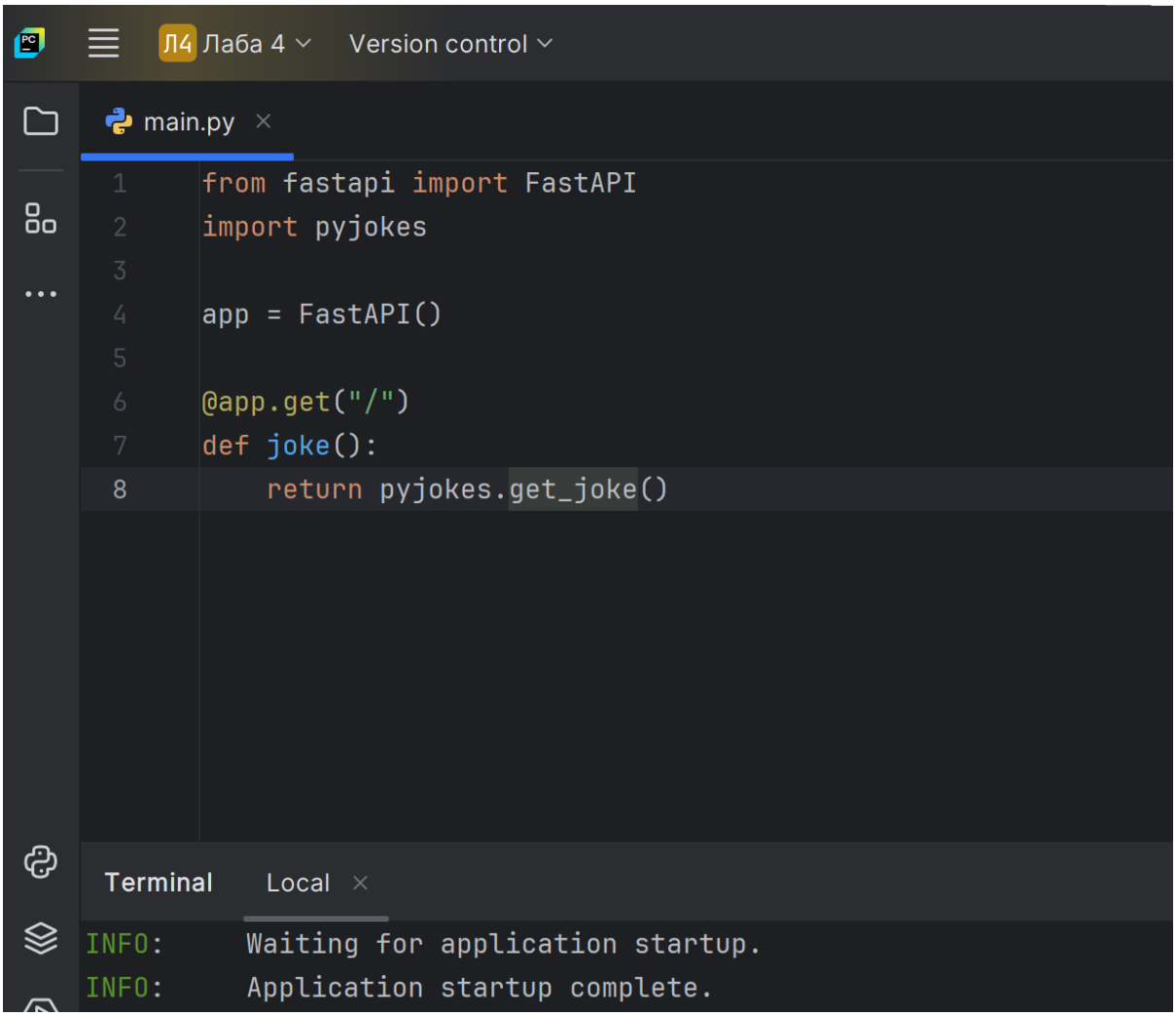
Цель работы:

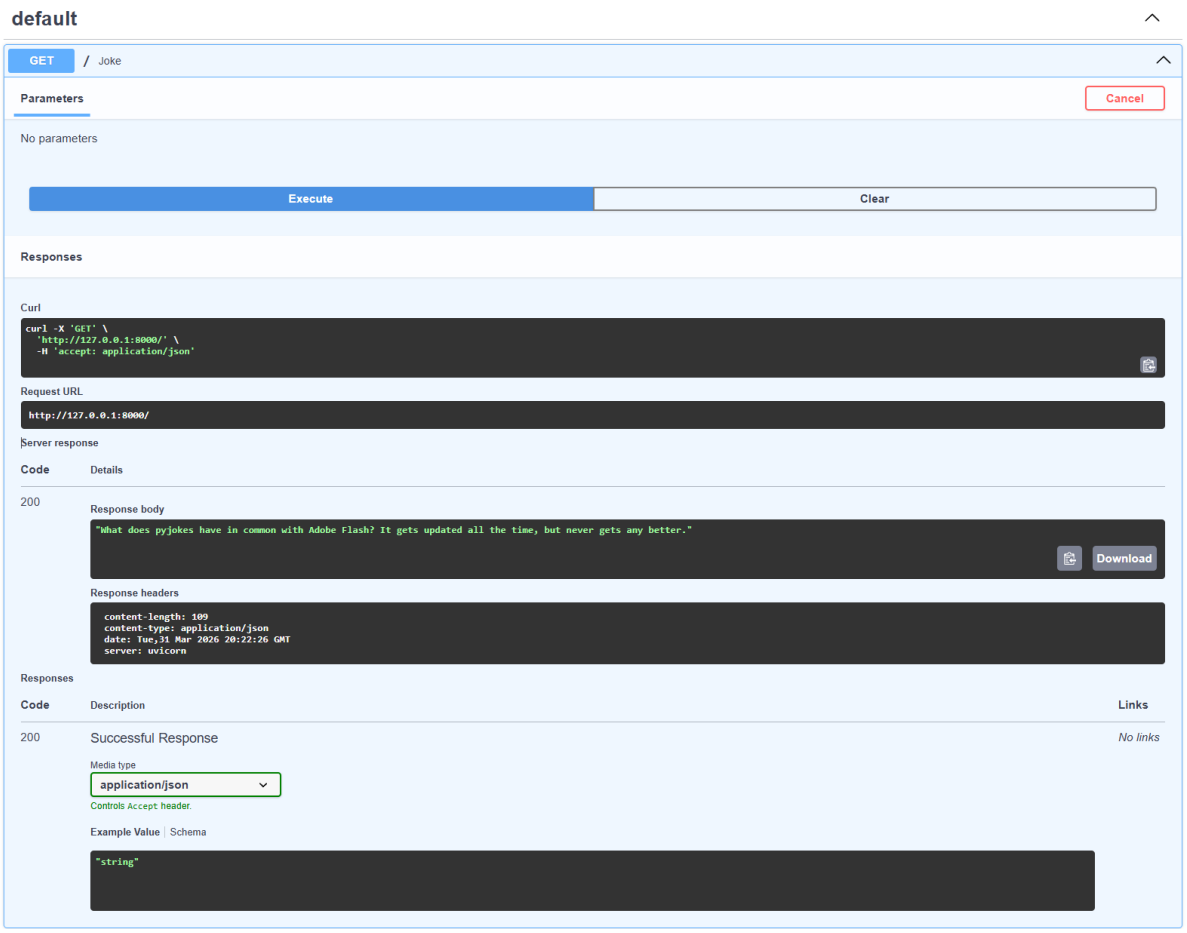
Задачи:

Изучить разработку RESTful API, научиться создавать свои api сервисы на fastapi и проводить валидацию данных при помощи pydantic.

Ход работы:

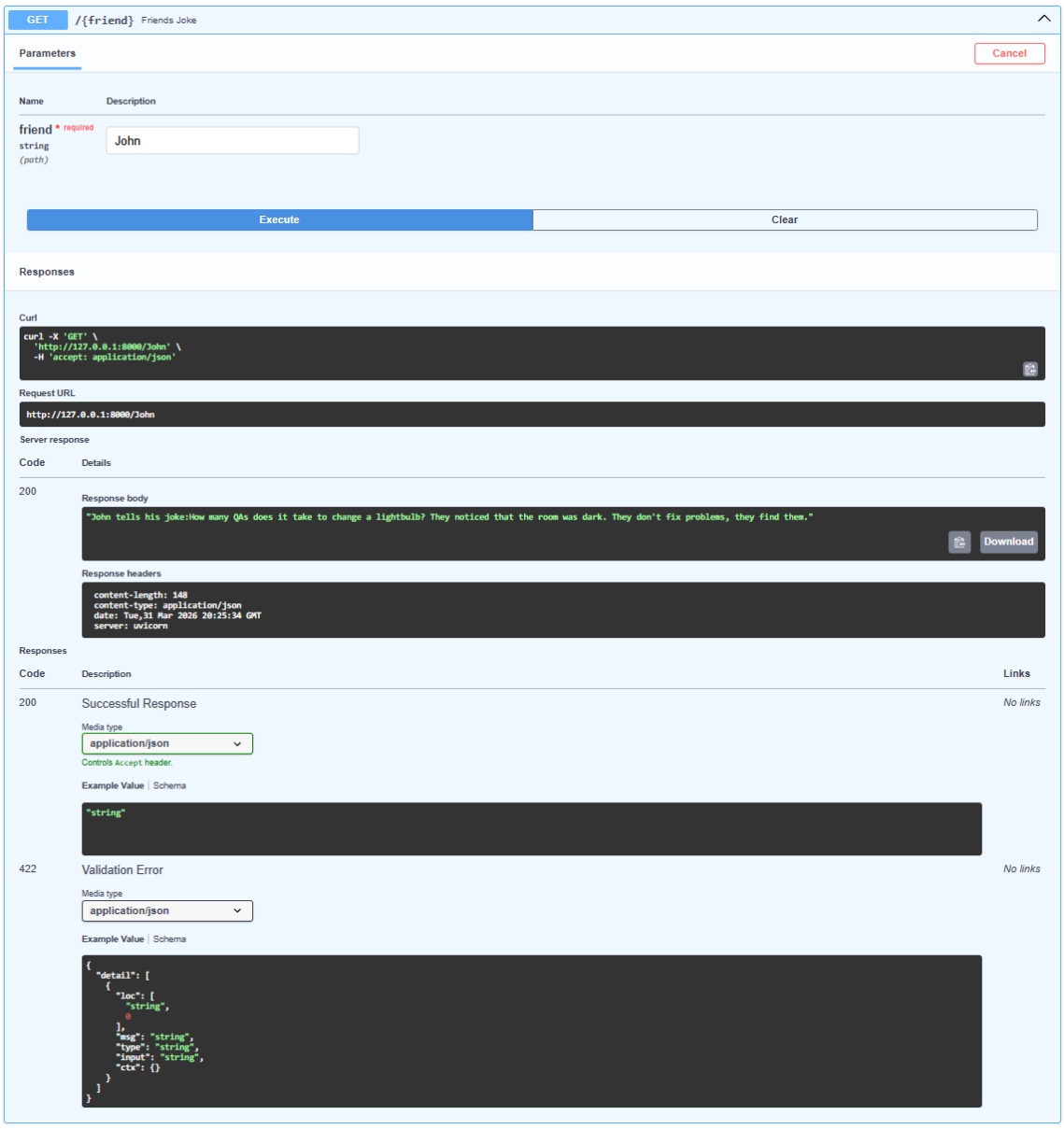
1. Создана основа программы с простым роутом.





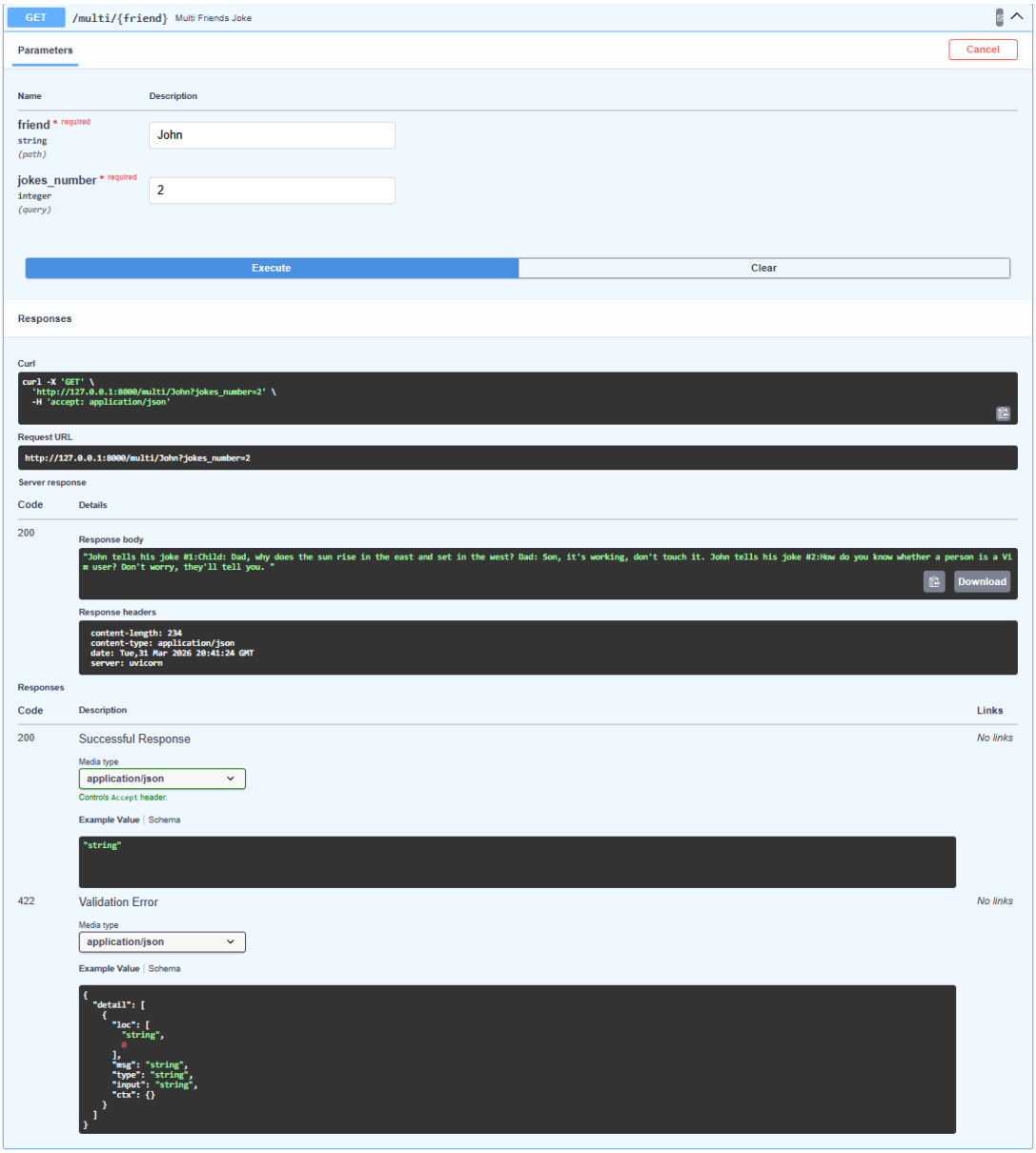
2. Добавлен еще один роут, где указан параметр в пути, чтобы была возможность представить шутку от какого-то конкретного человека. Для этого в фигурных скобках добавлено название желаемого параметра и добавлено оно же в параметрах функции.

```python
@app.get("/{friend}")
def friends_joke(friend: str):
    return friend + " tells his joke:" + pyjokes.get_joke()
```



3. Добавлен еще один роут, где есть возможность выбрать количество шуток. Для этого добавлен еще один параметр, который не указан в пути роута. Это query параметр jokes_number.

```python
@app.get("/multi/{friend}")
def multi_friends_joke(friend: str, jokes_number: int):
    result = ""
    for i in range(jokes_number):
        result += friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return result
```



4. Создан новый роут, используя метод POST, и создна схему тела запроса, которую будет принимать роут для корректной работы. Импортирован класс BaseModel из библиотеки pydantic и создана на его основе схема получения шутки, которую и передаем функции на вход

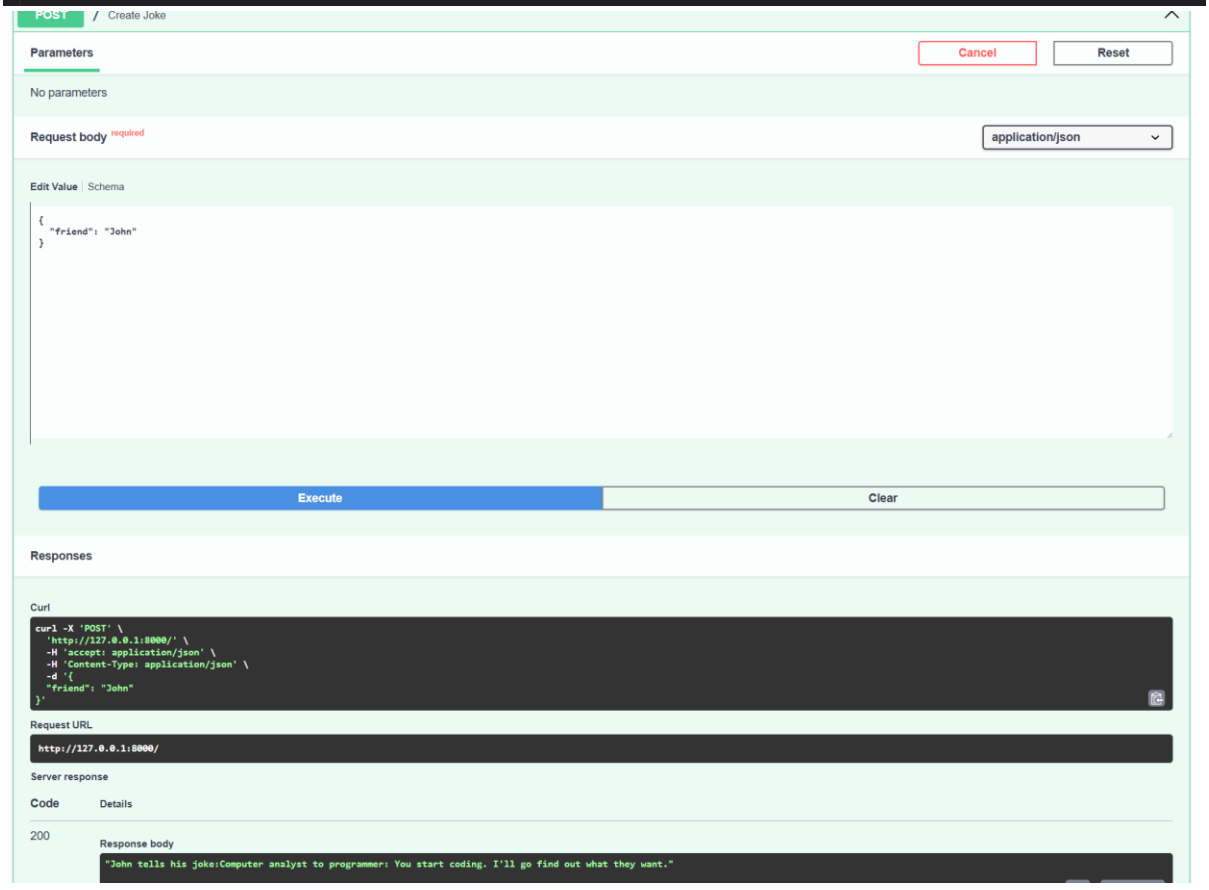
```python
from fastapi import FastAPI
import pyjokes
from pydantic import BaseModel

app = FastAPI()

class Joke(BaseModel):
    friend: str
    joke: str

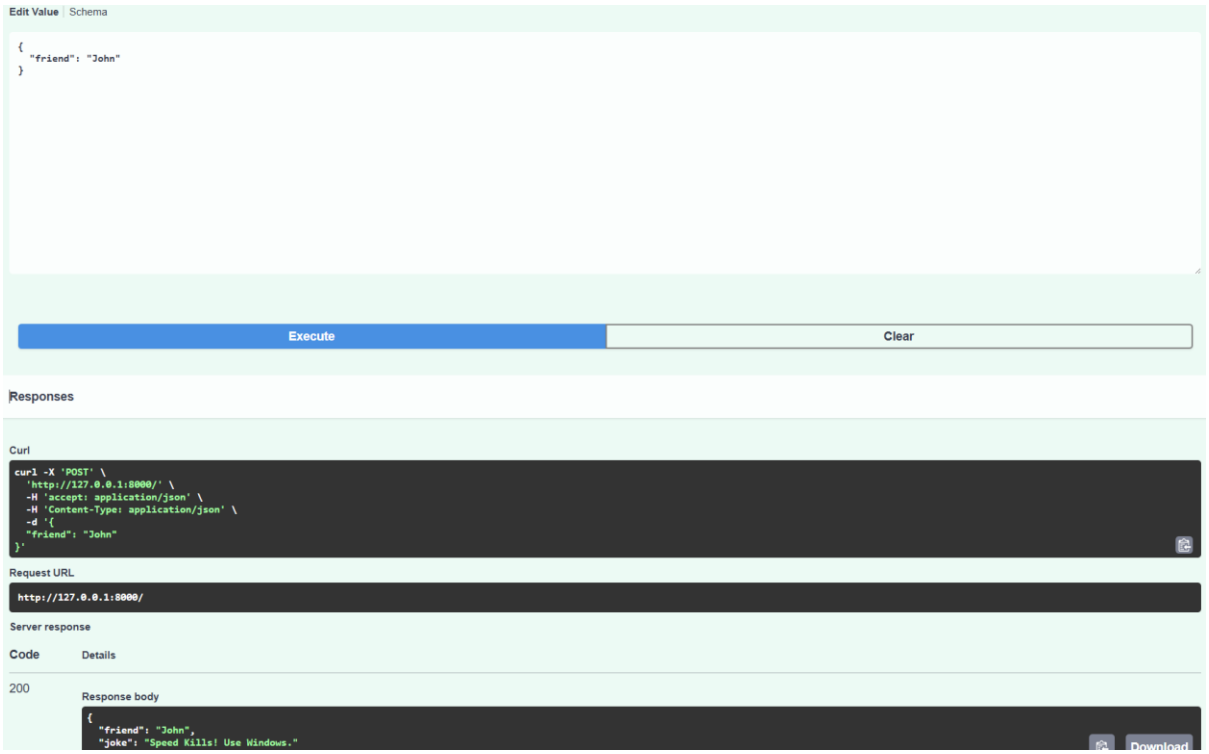
class JokeInput(BaseModel):  1 usage
    friend: str

@app.post("/")
def create_joke(joke_input: JokeInput):
    return joke_input.friend + " tells his joke:" + pyjokes.get_joke()
```



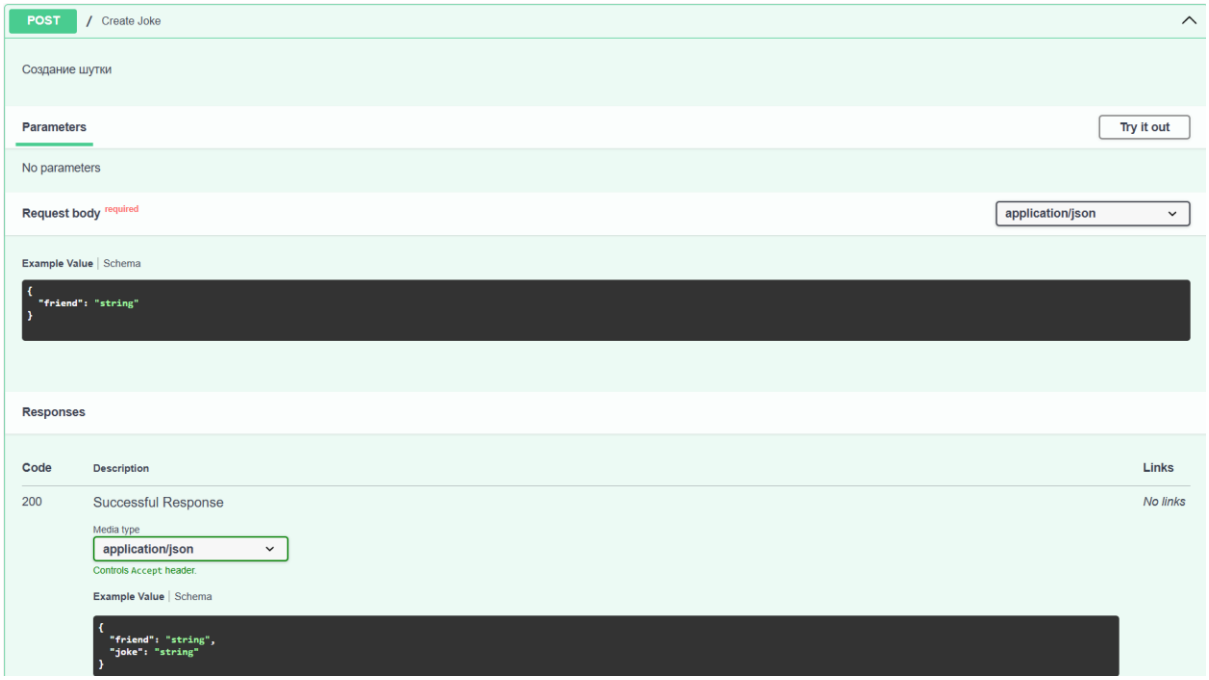
5. Обновлен ранее созданный POST метод, чтобы ответ выдавался в формате ранее описанным в схеме Joke. Обратим внимание, что схема не показалась в ожидаемом ответе.

```python
@app.post("/")
def create_joke(joke_input: JokeInput):
    return Joke(friend=joke_input.friend, joke=pyjokes.get_joke())
```



6. Добавлен комментарий с описанием роута и, чтобы добавить валидацию ответа функции, в параметрах поставлен response_model.

```python
@app.post(path: "/", response_model=Joke)
def create_joke(joke_input: JokeInput):
    """Создание шутки"""
    return Joke(friend=joke_input.friend, joke=pyjokes.get_joke())
```

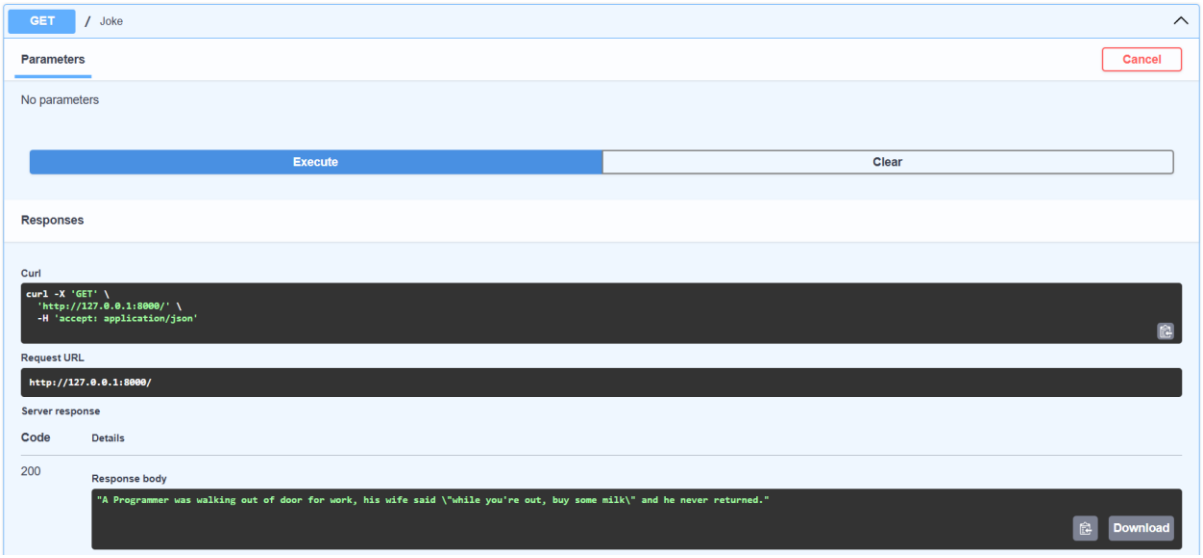


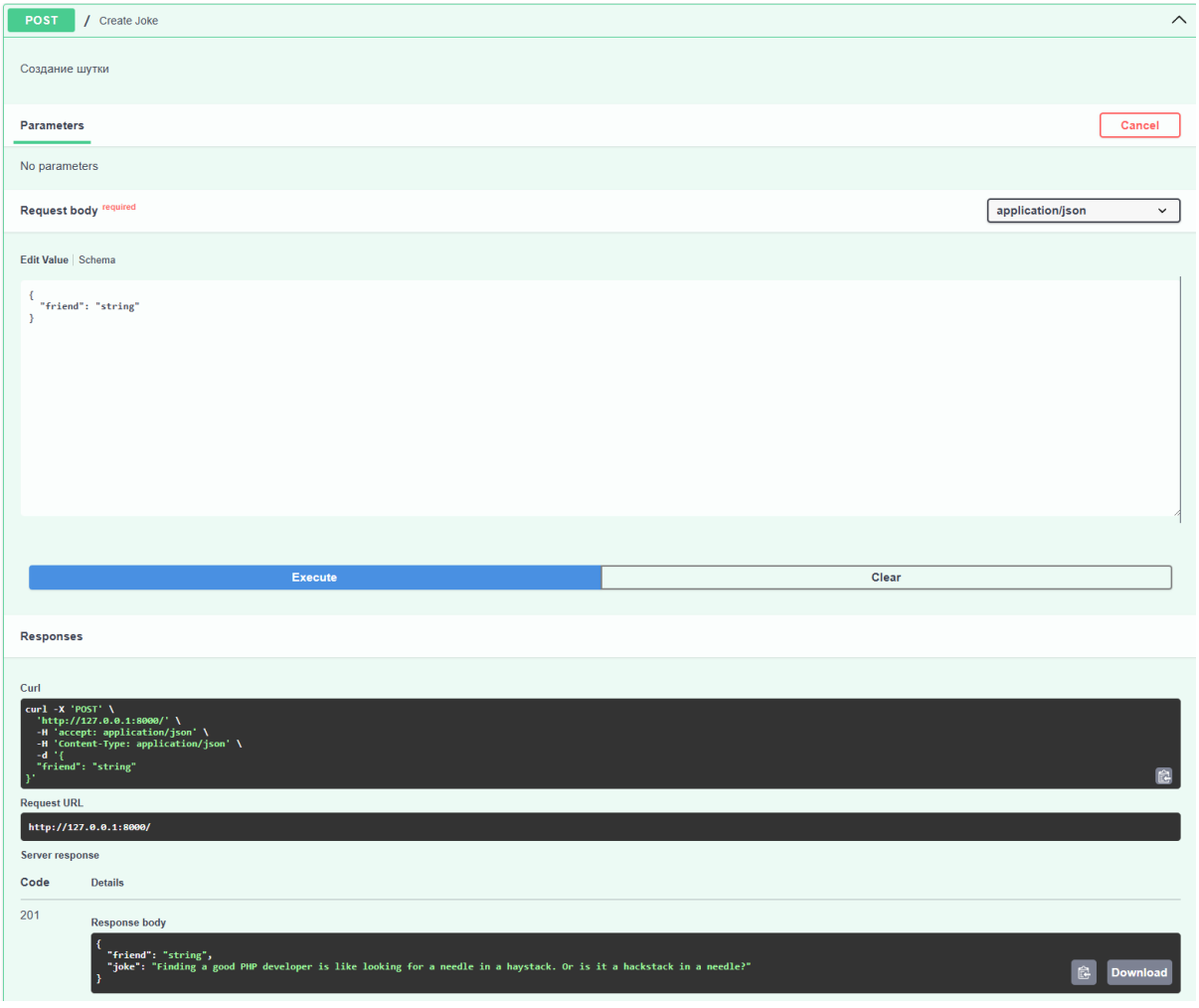
7. Модифицированы существующие эндпоинты таким образом, чтобы каждому из них был назначен корректный HTTP-статус ответа:

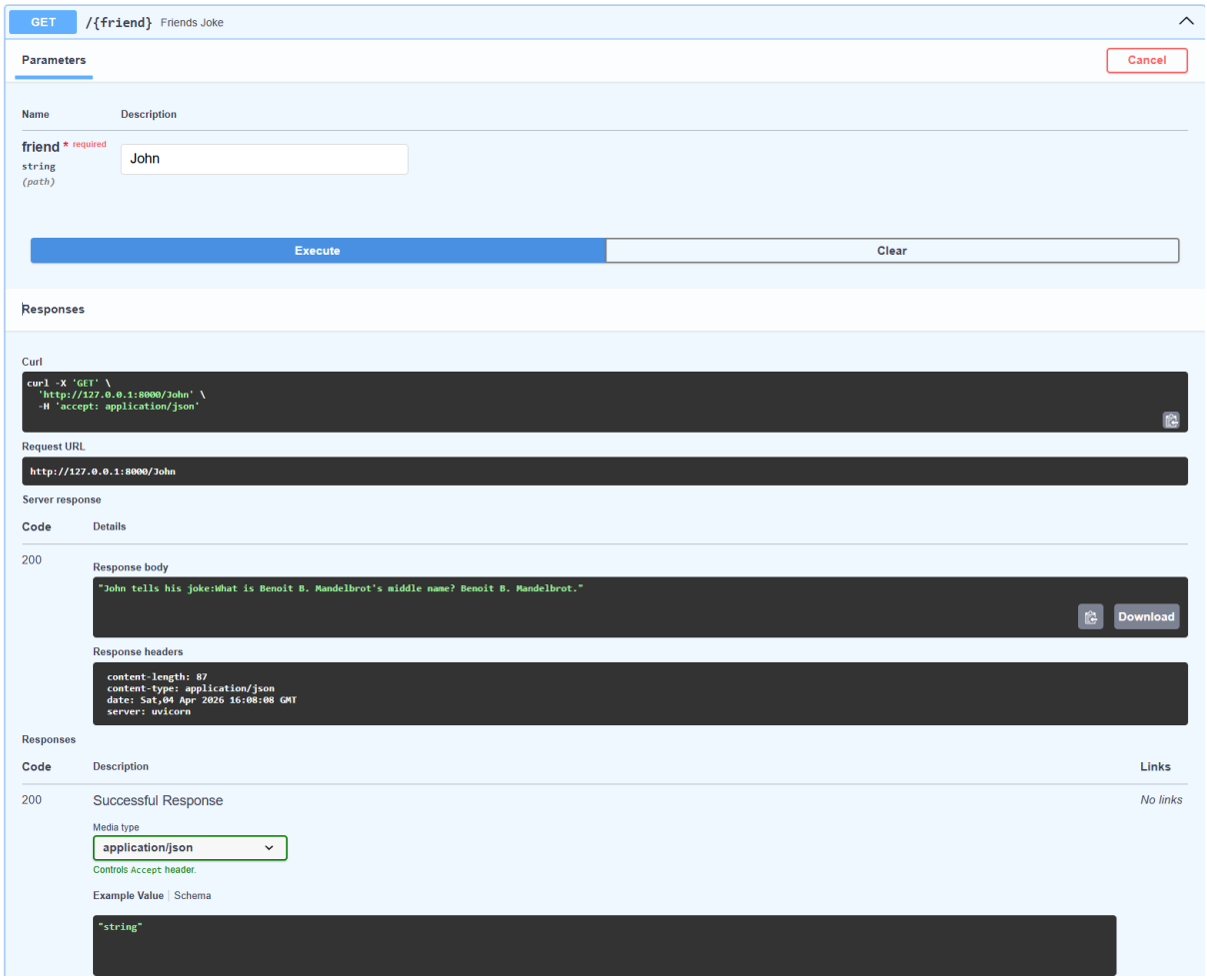
- GET – 200 OK
- POST – 201 Created

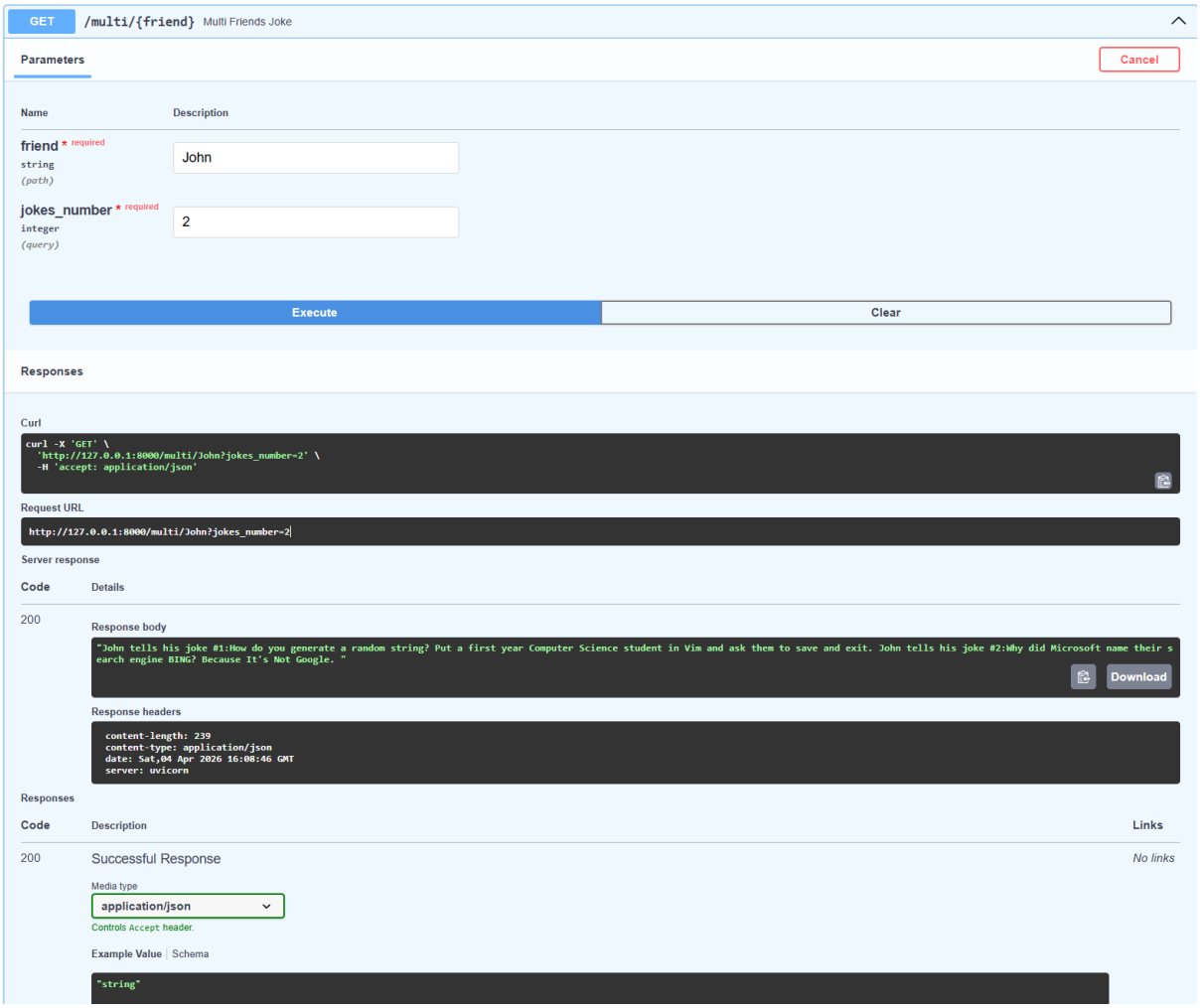
Проверена работа эндпоинтов через Swagger и Postman.

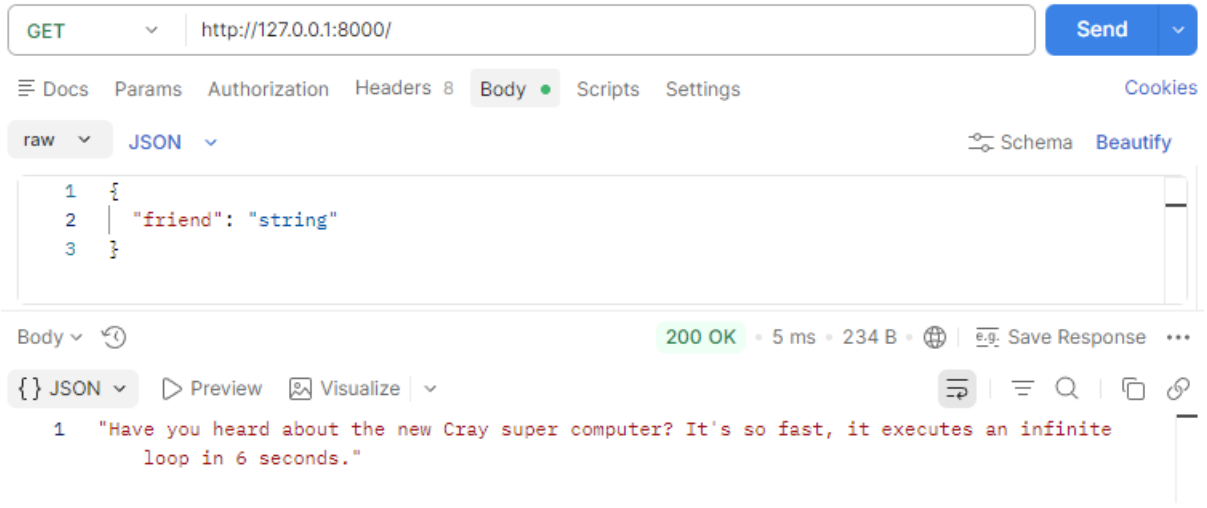
```python
@app.post(path: "/", status_code=201, response_model=Joke)
@app.get(path: "/", status_code=200)
@app.get(path: "/{friend}", status_code=200)
@app.get(path: "/multi/{friend}", status_code=200)
```

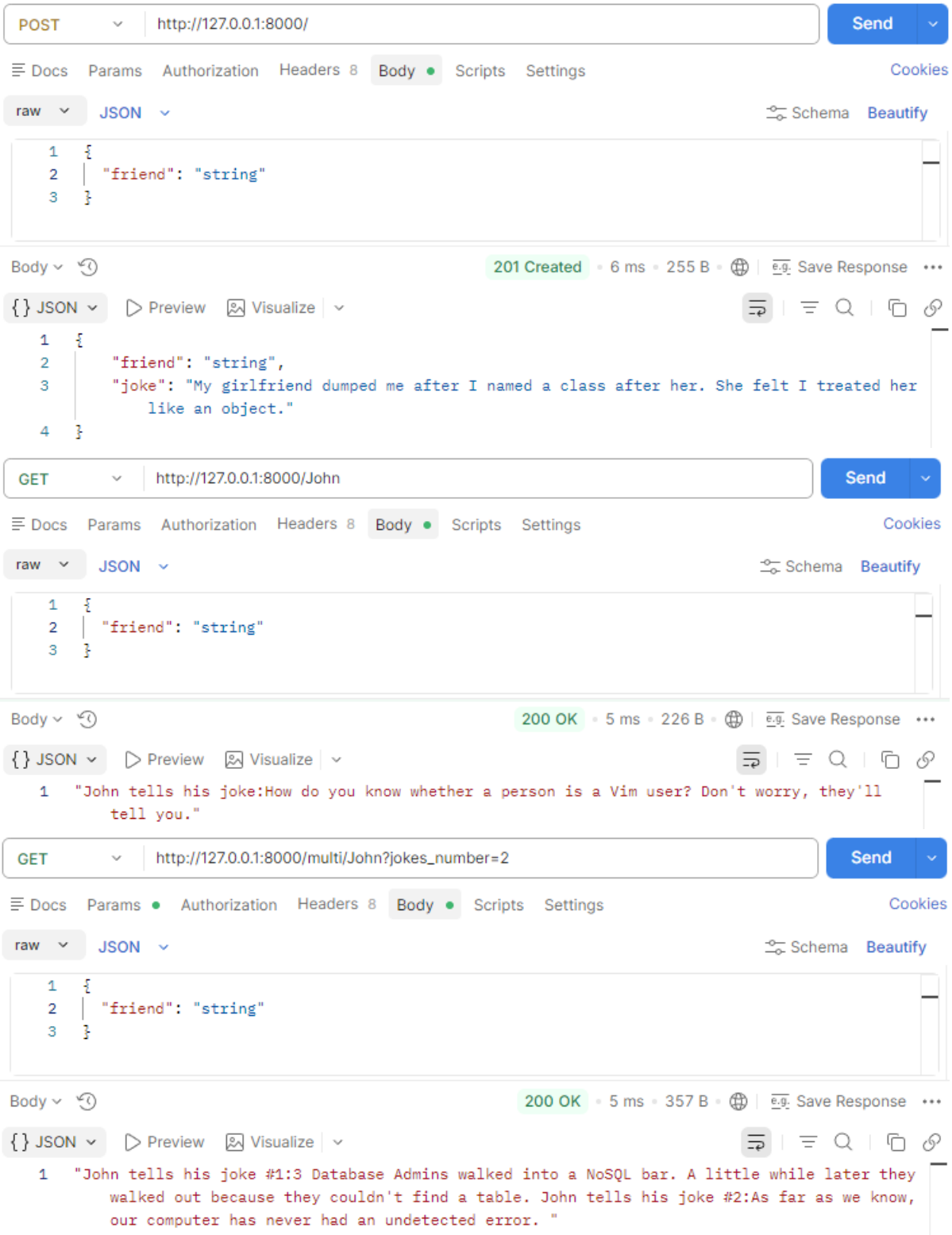

GET
/ Joke
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/
Server response
Code
Details
200
Response body
"A Programmer was walking out of door for work, his wife said \"while you're out, buy some milk\" and he never returned."
Download


POST
/ Create Joke
Создание шутки
Parameters
Cancel
No parameters
Request body
required
application/json
Edit Value
Schema
{
"friend": "string"
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"friend": "string"
}'
Request URL
http://127.0.0.1:8000/
Server response
Code
Details
201
Response body
{
"friend": "string",
"joke": "Finding a good PHP developer is like looking for a needle in a haystack. Or is it a hackstack in a needle?"
}
Download


GET
/{friend} Friends Joke
Parameters
Cancel
Name
Description
friend * required
string
(path)
John
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/John' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/John
Server response
Code
Details
200
Response body
"John tells his joke:What is Benoit B. Mandelbrot's middle name? Benoit B. Mandelbrot."
Download
Response headers
content-length: 87
content-type: application/json
date: Sat,04 Apr 2026 16:08:08 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"


GET
/multi/{friend} Multi Friends Joke
Parameters
Cancel
Name
Description
friend * required
string
(path)
John
jokes_number * required
integer
(query)
2
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/multi/John?jokes_number=2' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/multi/John?jokes_number=2
Server response
Code
Details
200
Response body
"John tells his joke #1:How do you generate a random string? Put a first year Computer Science student in Vim and ask them to save and exit. John tells his joke #2:Why did Microsoft name their search engine BING? Because It's Not Google. "
Download
Response headers
content-length: 239
content-type: application/json
date: Sat,04 Apr 2026 16:08:46 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value
Schema
"string"


GET
http://127.0.0.1:8000/
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
1 {
2 "friend": "string"
3 }
Body
200 OK
5 ms
234 B
Save Response
{} JSON
Preview
Visualize
1 "Have you heard about the new Cray super computer? It's so fast, it executes an infinite loop in 6 seconds."



8. Добавлены ограничения на входные параметры с использованием Pydantic:
   - минимальная длина строки для параметра friend
   - допустимый диапазон значений для параметра jokes_number

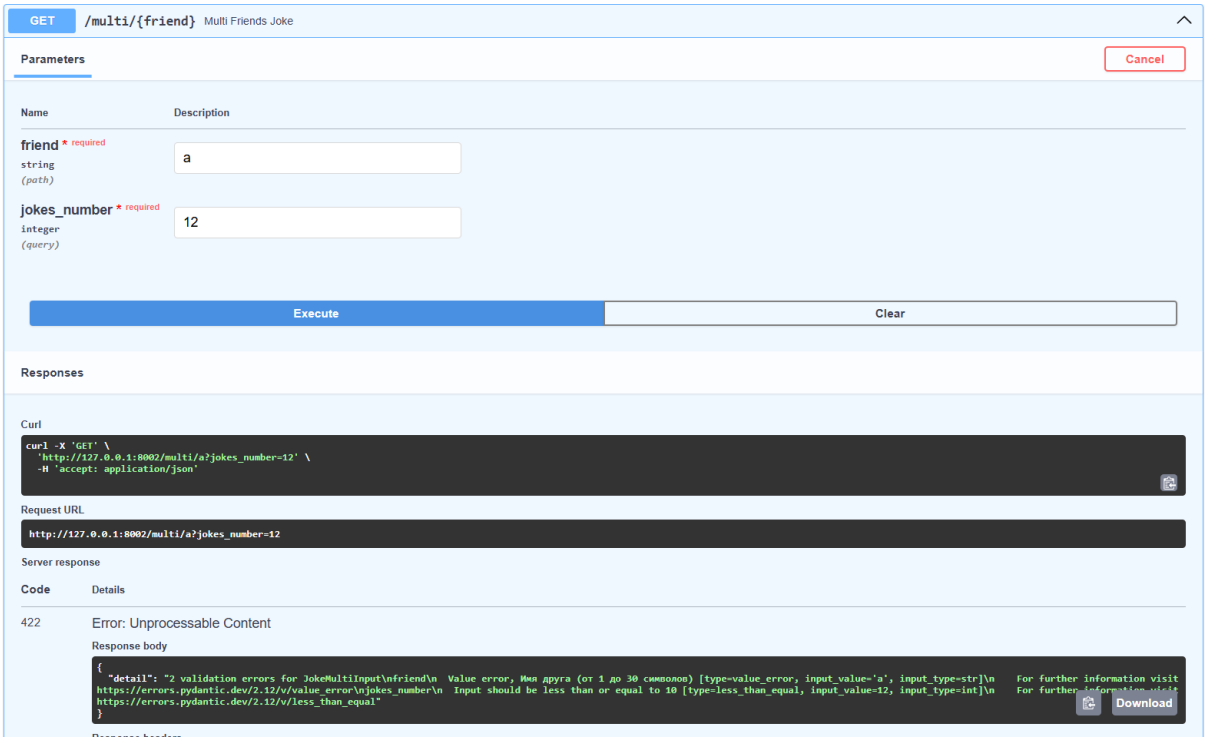
```python
class JokeMulti(BaseModel):  2 usages
    jokes: str
```

```python
class JokeMultiInput(BaseModel):  3 usages
    friend: str = Field(..., min_length=1, max_length=20, description="Имя друга (от 2 до 30 символов)")
    jokes_number: int = Field(..., ge=1, le=10, description="Кол-во шуток (от 1 до 10)")

    @field_validator('friend')
    @classmethod
    def validate_friend_name(cls, v):
        if len(v.strip()) < 2 or len(v.strip()) > 30:
            raise ValueError('Имя друга (от 2 до 30 символов)')
        return v.strip()

    @field_validator('jokes_number')
    @classmethod
    def validate_jokes_number(cls, v):
        if not v or v > 10:
            raise ValueError('Кол-во шуток (от 1 до 10)')
        return v
```

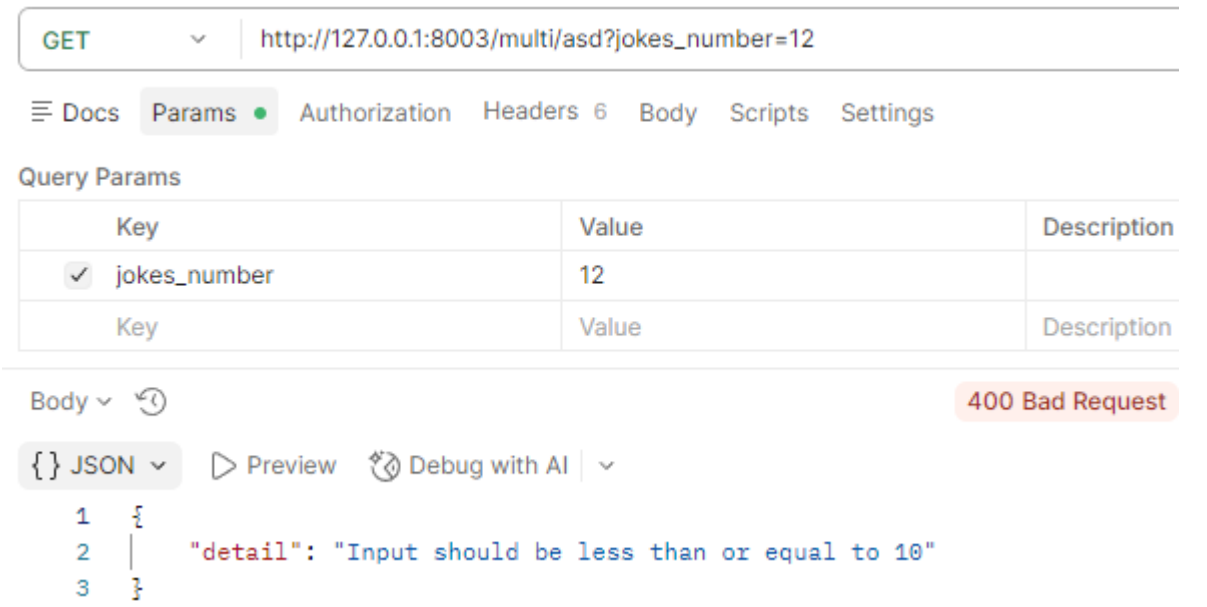
```python
@app.get( path: "/multi/{friend}", status_code=200)
def multi_friends_joke(friend: str, jokes_number: int):
    try:
        validated_data = JokeMultiInput(friend=friend, jokes_number=jokes_number)
    except Exception as e:
        raise HTTPException(status_code=422, detail=str(e))
    result = ""
    for i in range(validated_data.jokes_number):
        result += validated_data.friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return JokeMulti(jokes=result)
```

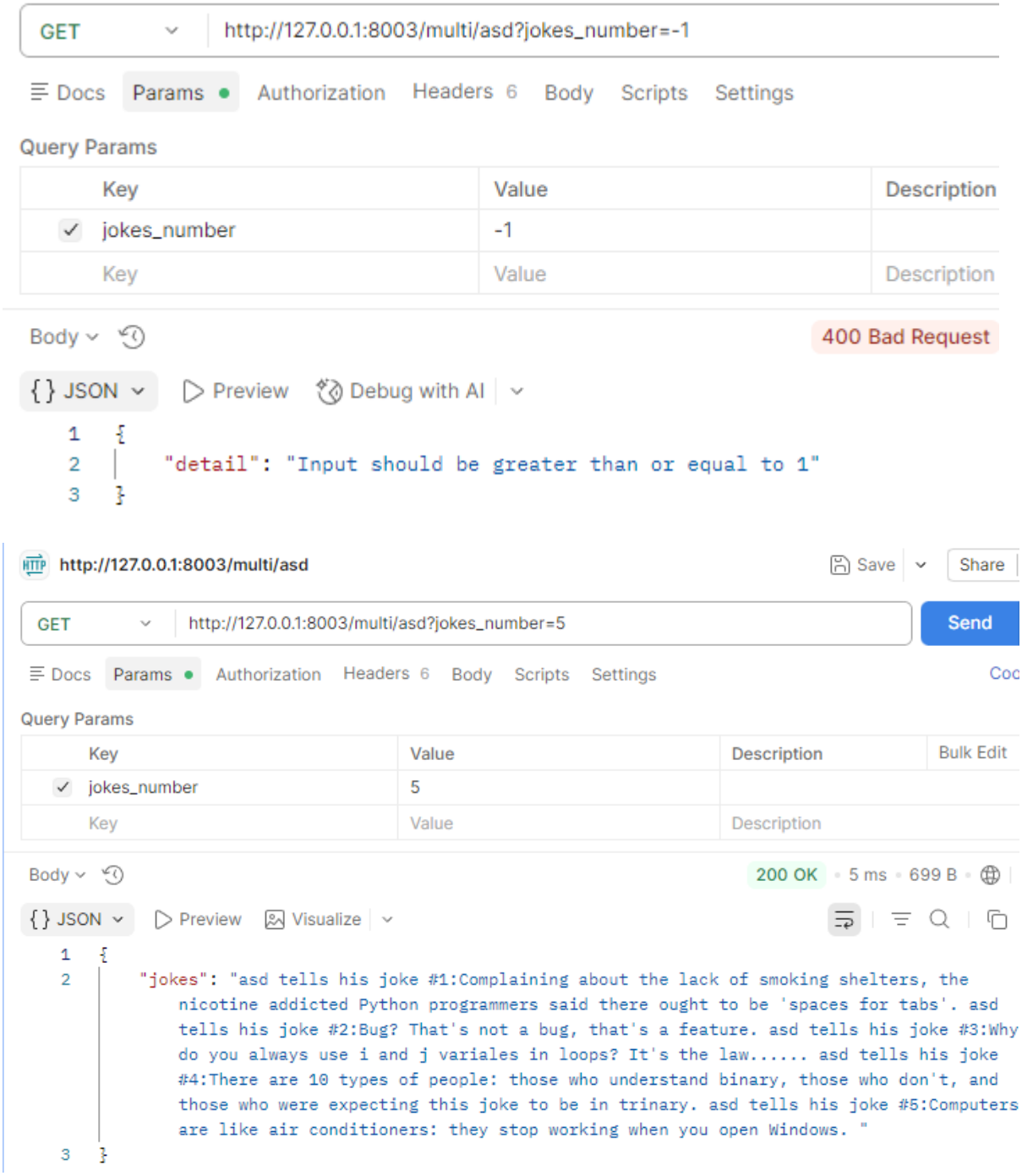


Теперь, при подаче некорректных данных в параметры, сервер выдает ошибку 422. Это означает, что сервер понял запрос и его синтаксис верен, но содержание имеет семантические ошибки.

9. Реализована проверка параметра jokes_number. В случае передачи недопустимого значения возвращается ошибка 400 Bad Request с пояснением причины. Проверкна работа эндпоинта при корректных и некорректных данных.

```python
@app.get( path: "/multi/{friend}", status_code=200)
def multi_friends_joke(friend: str, jokes_number: int):
    try:
        validated_data = JokeMultiInput(friend=friend, jokes_number=jokes_number)
    except ValidationError as e:
        if e.errors()[0]['type'] == 'greater_than_equal' or e.errors()[0]['type'] == 'less_than_equal':
            raise HTTPException(status_code=400, detail=e.errors()[0]['msg'])
        else:
            raise HTTPException(status_code=422, detail=e.errors()[0]['msg'])
    result = ""
    for i in range(validated_data.jokes_number):
        result += validated_data.friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return JokeMulti(jokes=result)
```



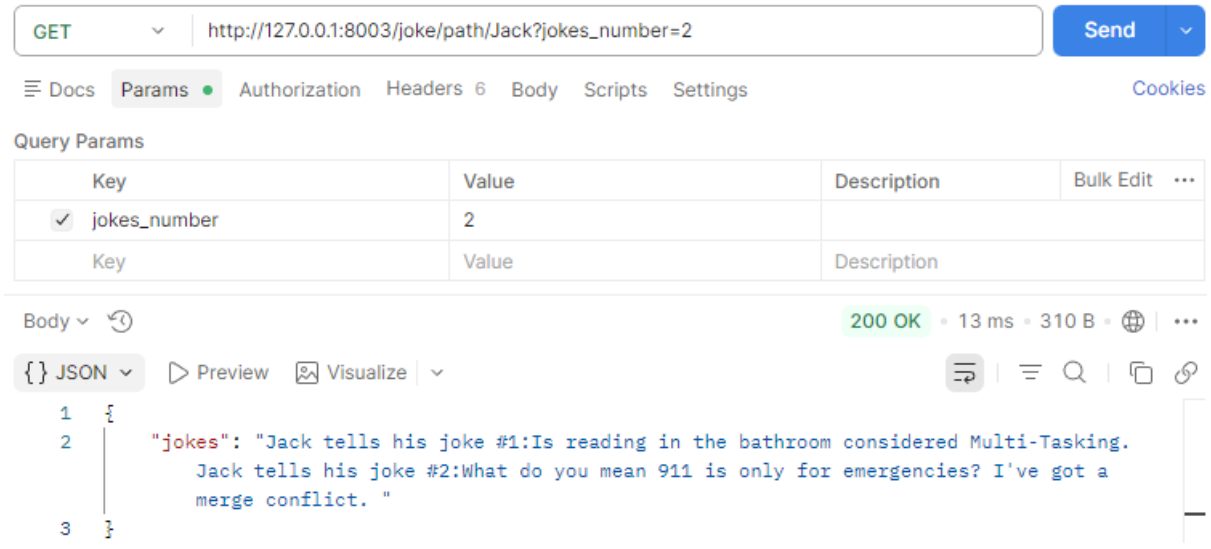


10. Реализованы два эндпоинта с одинаковой логикой:
    - один с использованием path-параметр
    - один с использованием query-параметра

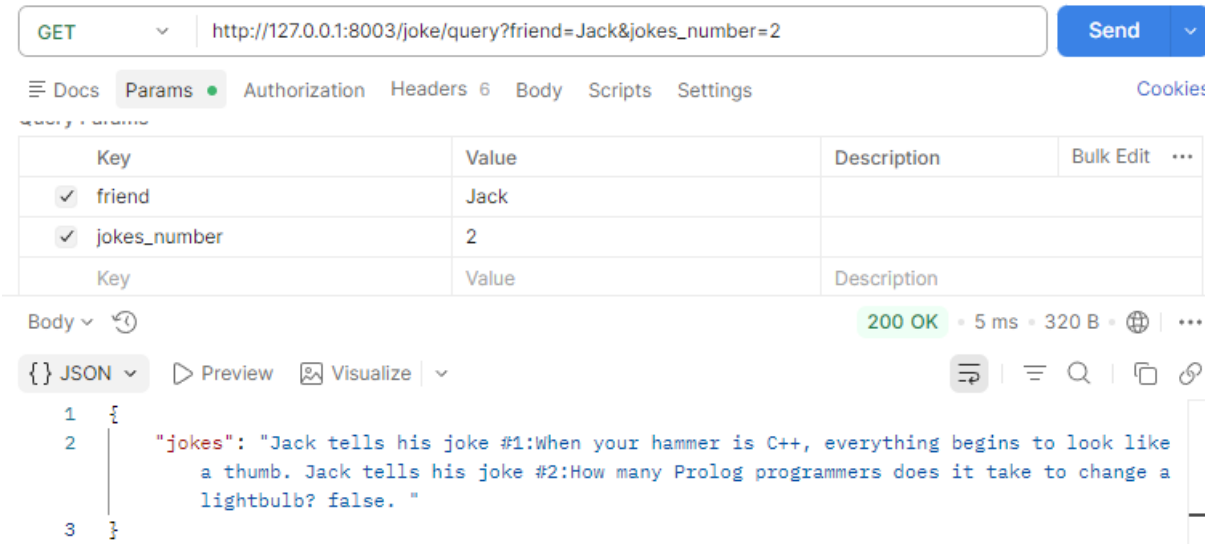
    Сделан вывод о различии между ними: path параметры являются обязательными, в отличии от query, им нельзя задать значение по умолчанию и их порядок фиксирован. По этим причинам path параметры лучше всего

использовать для идентификации ресурсов, в то время как query параметры больше подходят для операций фильтрации и указания дополнительных параметров запросов.

```python
@app.get("/joke/path/{friend}")
def get_joke_path(friend: Annotated[str, Path(...)], jokes_number: Annotated[int, Query(...)]):
    try:
        validated_data = JokeMultiInput(friend=friend, jokes_number=jokes_number)
    except ValidationError as e:
        res = []
        for mistake in e.errors():
            res.append(mistake['msg'])
        raise HTTPException(status_code=422, detail=res)
    result = ""
    for i in range(validated_data.jokes_number):
        result += validated_data.friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return JokeMulti(jokes=result)
```



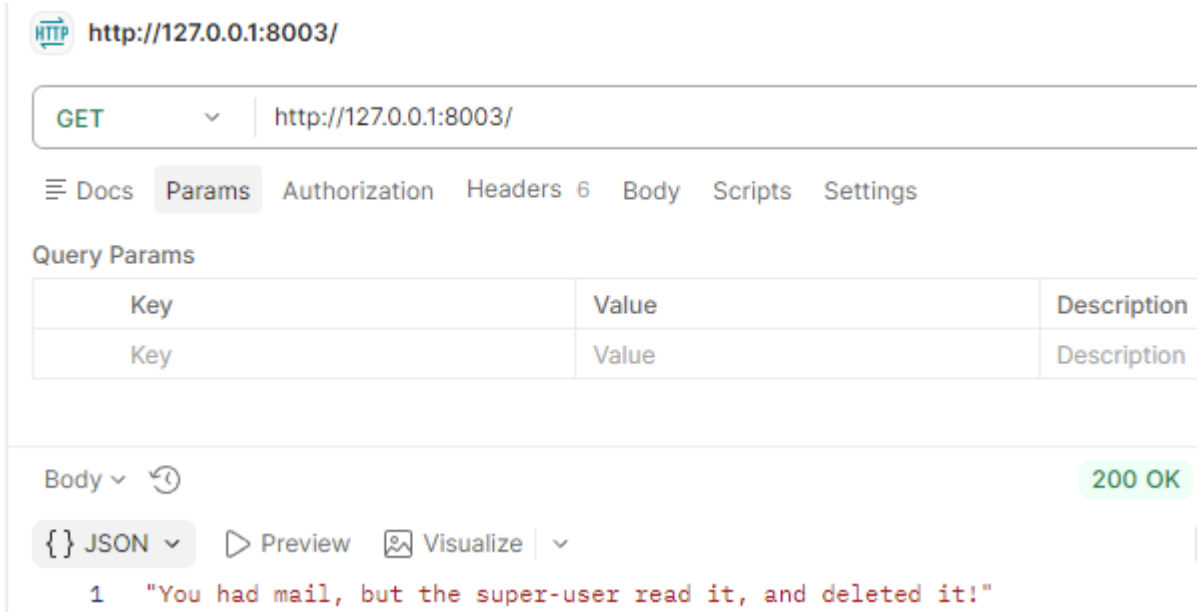
```python
@app.get("/joke/query")
def get_joke_path(friend: Annotated[str, Query(...)], jokes_number: Annotated[int, Query(...)]):
    try:
        validated_data = JokeMultiInput(friend=friend, jokes_number=jokes_number)
    except ValidationError as e:
        res = []
        for mistake in e.errors():
            res.append(mistake['msg'])
        raise HTTPException(status_code=422, detail=res)
    result = ""
    for i in range(validated_data.jokes_number):
        result += validated_data.friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return JokeMulti(jokes=result)
```



11. Протестированы все реализованные эндпоинты с использованием Postman. Для каждого запроса:
    - Зафиксирован метод и URL
    - Указан HTTP-статус ответа
    - Сохранен фрагмент ответа сервера

Включены скриншоты корректных и ошибочных запросов.

```python
@app.get( path: "/", status_code=200)
def joke():
    return pyjokes.get_joke()
```




```python
@app.post( path: "/", status_code=201, response_model=Joke)
def create_joke(joke_input: JokeInput):
    """Создание шутки"""
    return Joke(friend=joke_input.friend, joke=pyjokes.get_joke())
```

```python
class Joke(BaseModel):
    friend: str
    joke: str
```

```python
class JokeInput(BaseModel):  1 usage
    friend: str = Field(..., min_length=1, max_length=20, description="Имя друга (от 1 до 30 символов)")
```



http://127.0.0.1:8003/ Save Share

POST http://127.0.0.1:8003/ Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
{

}
```

Body 422 Unprocessable Content 6 ms 233 B

{} JSON Preview Debug with AI


```
{
    "detail": [
        {
            "type": "missing",
            "loc": [
                "body",
                "friend"
            ],
            "msg": "Field required",
            "input": {}
        }
    ]
}
```


```python
@app.get( path: "/{friend}", status_code=200)
def friends_joke(friend: str):
    return friend + " tells his joke:" + pyjokes.get_joke()
```

GET http://127.0.0.1:8003/Jack Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
{

}
```

Body 200 OK 12 ms 274 B

{} JSON Preview Visualize


```
"Jack tells his joke:An SEO expert walks into a bar, bars, pub, public house, Irish pub, tavern, bartender, beer, liquor, wine, alcohol, spirits..."
```

```python
@app.get( path: "/multi/{friend}", status_code=200)
def multi_friends_joke(friend: str, jokes_number: int):
    try:
        validated_data = JokeMultiInput(friend=friend, jokes_number=jokes_number)
    except ValidationError as e:
        if e.errors()[0]['type'] == 'greater_than_equal' or e.errors()[0]['type'] == 'less_than_equal':
            raise HTTPException(status_code=400, detail=e.errors()[0]['msg'])
        else:
            raise HTTPException(status_code=422, detail=e.errors()[0]['msg'])
    result = ""
    for i in range(validated_data.jokes_number):
        result += validated_data.friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return JokeMulti(jokes=result)
```

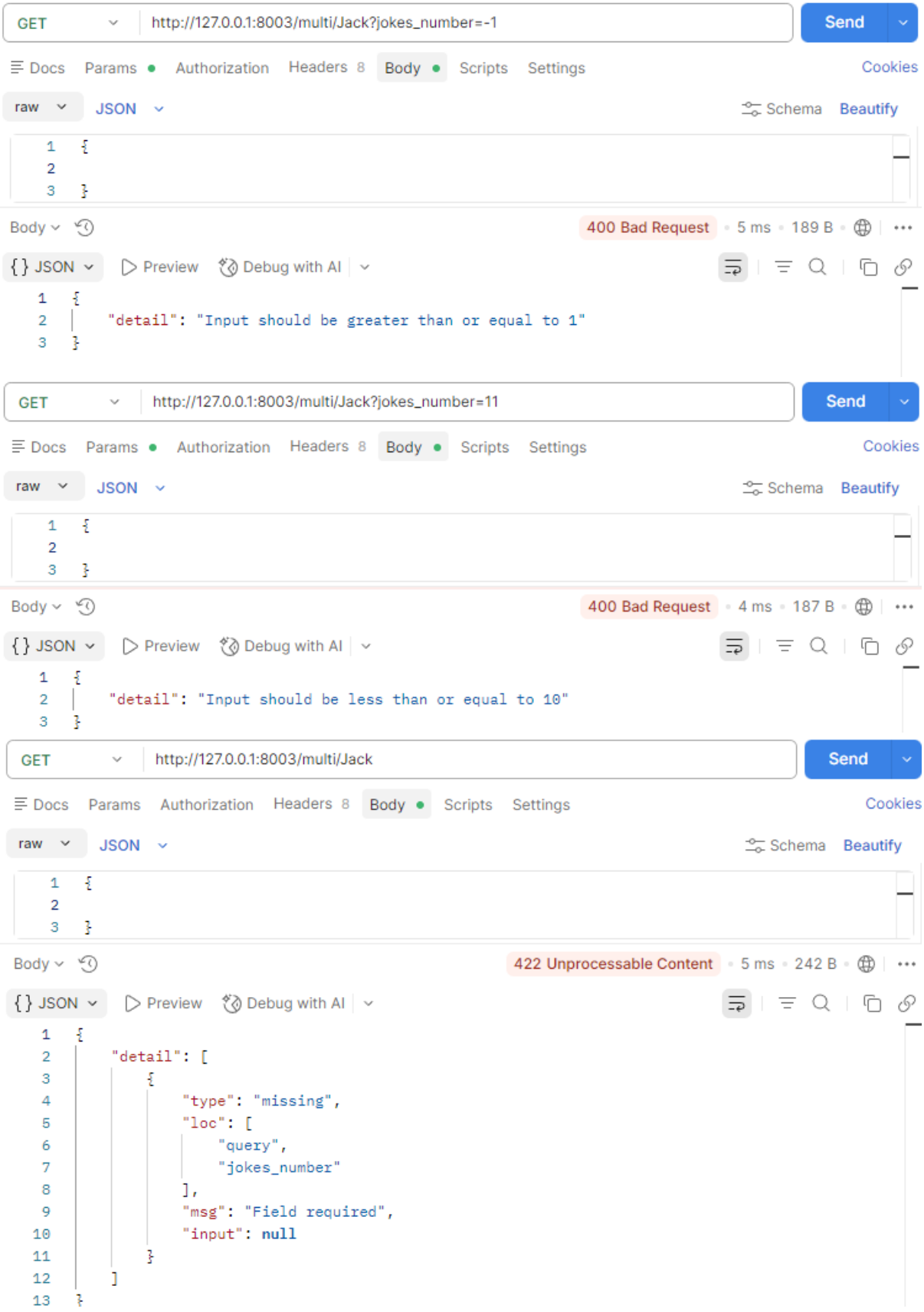
```python
class JokeMulti(BaseModel):
    jokes: str
```

```python
class JokeMultiInput(BaseModel):  3 usages
    friend: str = Field(..., min_length=1, max_length=20, description="Имя друга (от 2 до 30 символов)")
    jokes_number: int = Field(..., ge=1, le=10, description="Кол-во шуток (от 1 до 10)")

    @field_validator('friend')
    @classmethod
    def validate_friend_name(cls, v):
        if len(v.strip()) < 2 or len(v.strip()) > 30:
            raise ValueError('Имя друга (от 2 до 30 символов)')
        return v.strip()

    @field_validator('jokes_number')
    @classmethod
    def validate_jokes_number(cls, v):
        if not v or v > 10:
            raise ValueError('Кол-во шуток (от 1 до 10)')
        return v
```




GET
http://127.0.0.1:8003/multi/Jack?jokes_number=-1
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON Schema Beautify
{
}
Body 400 Bad Request 5 ms 189 B
{ } JSON Preview Debug with AI
{
"detail": "Input should be greater than or equal to 1"
}
GET
http://127.0.0.1:8003/multi/Jack?jokes_number=11
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON Schema Beautify
{
}
Body 400 Bad Request 4 ms 187 B
{ } JSON Preview Debug with AI
{
"detail": "Input should be less than or equal to 10"
}
GET
http://127.0.0.1:8003/multi/Jack
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON Schema Beautify
{
}
Body 422 Unprocessable Content 5 ms 242 B
{ } JSON Preview Debug with AI
{
"detail": [
{
"type": "missing",
"loc": [
"query",
"jokes_number"
],
"msg": "Field required",
"input": null
}
]
}

GET http://127.0.0.1:8003/multi/a?jokes_number=11 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
{

}
```

Body 422 Unprocessable Content 13 ms 221 B

{} JSON Preview Debug with AI

```
{
    "detail": "Value error, Имя друга (от 2 до 30 символов)"
}
```

```python
@app.get("/joke/path/{friend}")
def get_joke_path(friend: Annotated[str, Path(...)], jokes_number: Annotated[int, Query(...)]):
    try:
        validated_data = JokeMultiInput(friend=friend, jokes_number=jokes_number)
    except ValidationError as e:
        res = []
        for mistake in e.errors()['msg']:
            res.append(mistake)
        raise HTTPException(status_code=422, detail=res)
    result = ""
    for i in range(validated_data.jokes_number):
        result += validated_data.friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return JokeMulti(jokes=result)
```

GET http://127.0.0.1:8003/joke/path/Jack?jokes_number=2 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

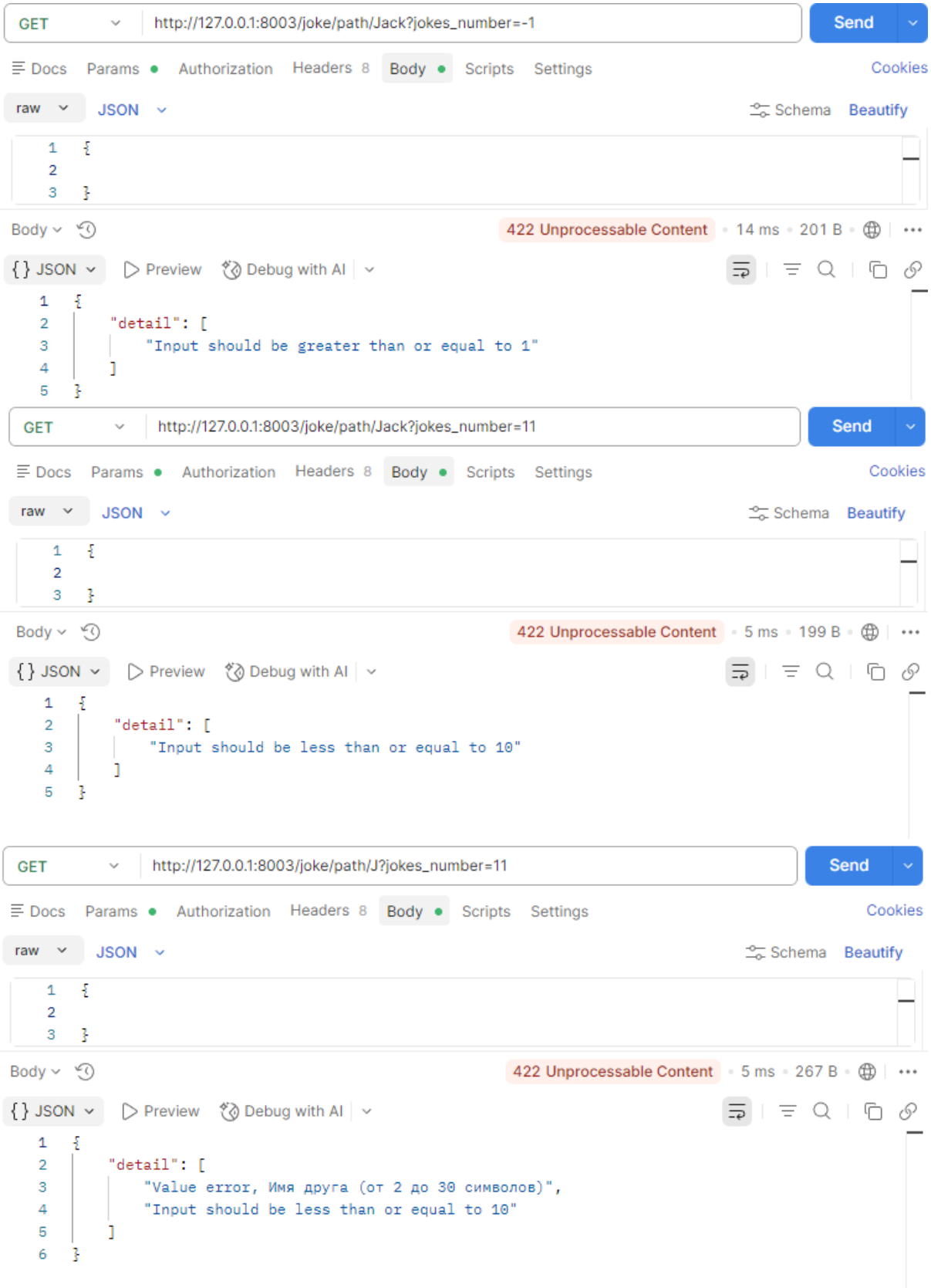
```
{

}
```

Body 200 OK 7 ms 379 B

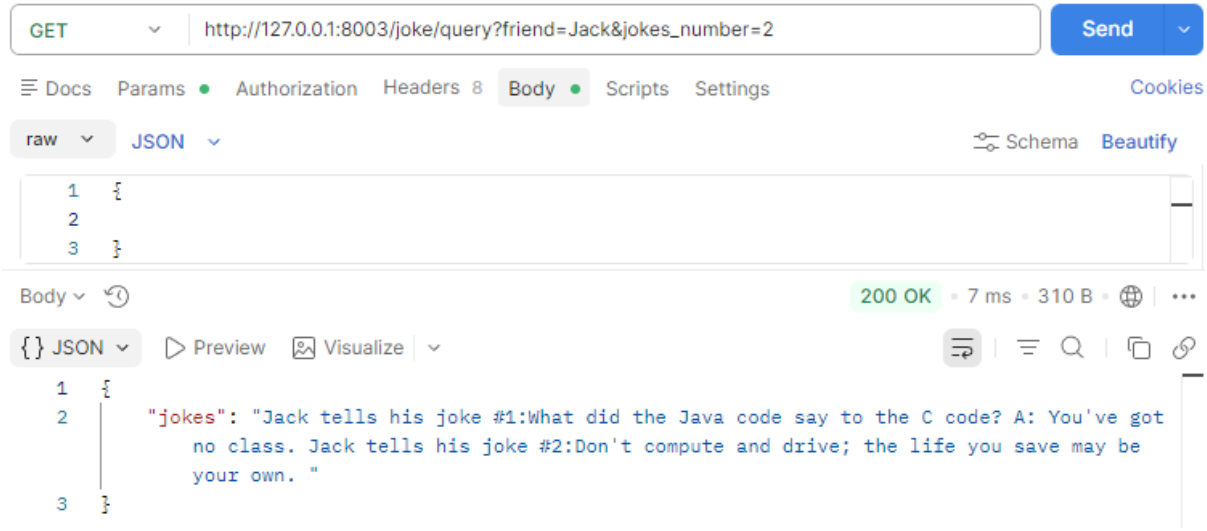
{} JSON Preview Visualize

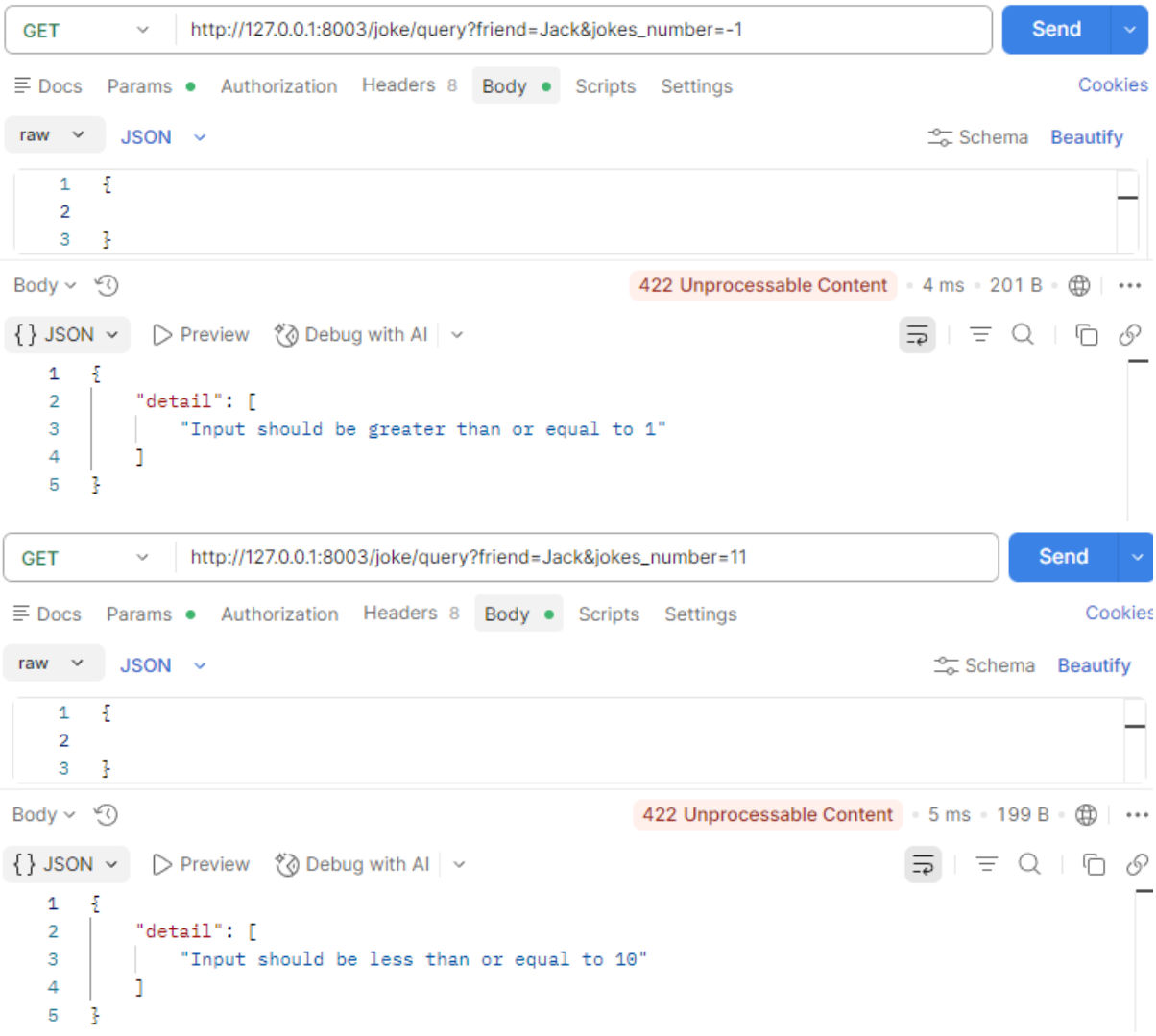
```
{
    "jokes": "Jack tells his joke #1:Debugging is like being the detective in a crime drama
        where you are also the murderer. Jack tells his joke #2:Finding a good PHP developer
        is like looking for a needle in a haystack. Or is it a hackstack in a needle? "
}
```


GET
http://127.0.0.1:8003/joke/path/Jack?jokes_number=-1
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON Schema Beautify
{
}
Body
422 Unprocessable Content 14 ms 201 B
{ } JSON Preview Debug with AI
{
"detail": [
"Input should be greater than or equal to 1"
]
}
GET
http://127.0.0.1:8003/joke/path/Jack?jokes_number=11
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON Schema Beautify
{
}
Body
422 Unprocessable Content 5 ms 199 B
{ } JSON Preview Debug with AI
{
"detail": [
"Input should be less than or equal to 10"
]
}
GET
http://127.0.0.1:8003/joke/path/J?jokes_number=11
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
raw JSON Schema Beautify
{
}
Body
422 Unprocessable Content 5 ms 267 B
{ } JSON Preview Debug with AI
{
"detail": [
"Value error, Имя друга (от 2 до 30 символов)",
"Input should be less than or equal to 10"
]
}

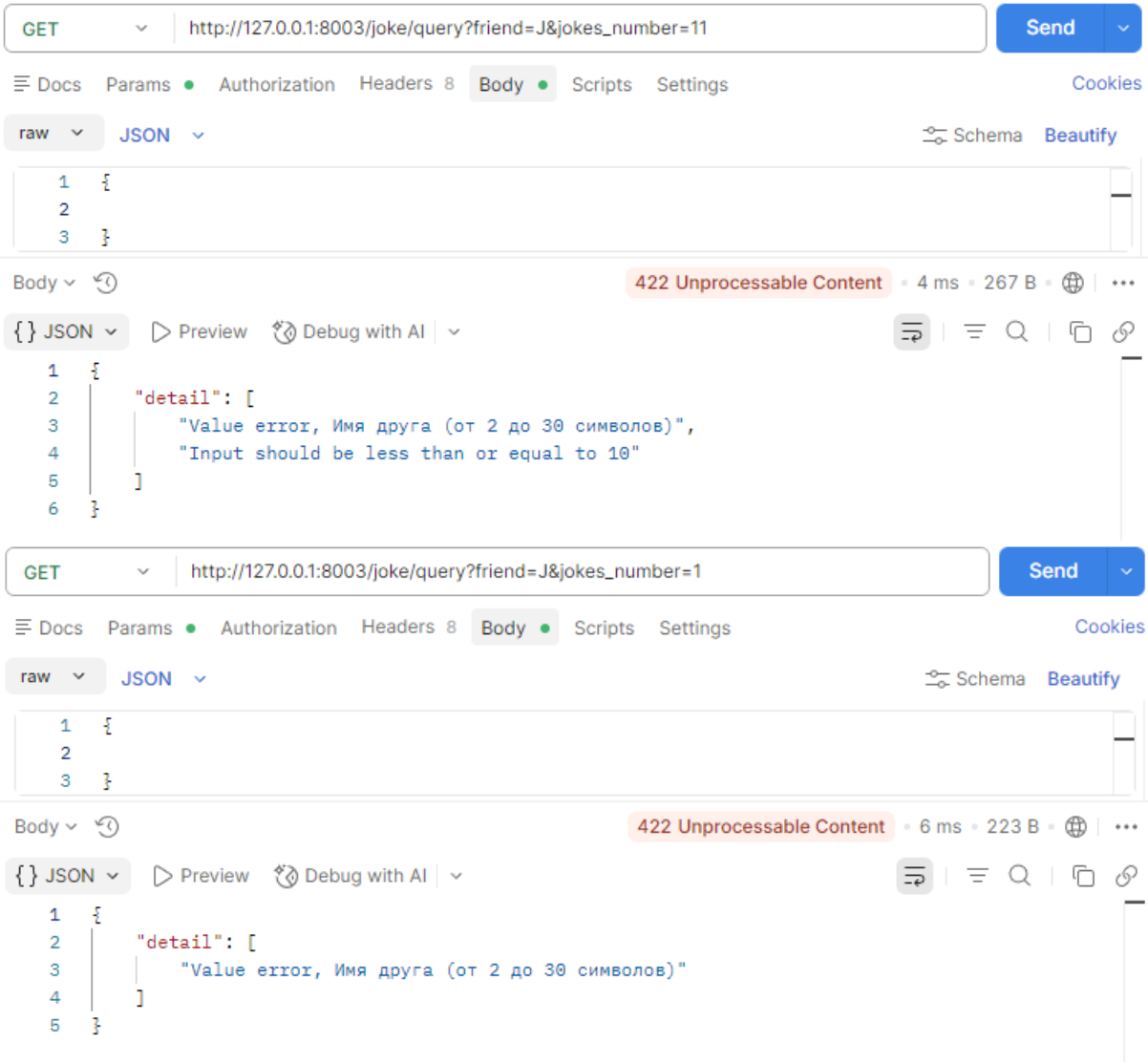



```python
@app.get("/joke/query")
def get_joke_path(friend: Annotated[str, Query(...)], jokes_number: Annotated[int, Query(...)]):
    try:
        validated_data = JokeMultiInput(friend=friend, jokes_number=jokes_number)
    except ValidationError as e:
        res = []
        for mistake in e.errors():
            res.append(mistake['msg'])
        raise HTTPException(status_code=422, detail=res)
    result = ""
    for i in range(validated_data.jokes_number):
        result += validated_data.friend + f" tells his joke #{i + 1}:" + pyjokes.get_joke() + " "
    return JokeMulti(jokes=result)
```




GET
http://127.0.0.1:8003/joke/query?friend=Jack&jokes_number=-1
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
}
Body
422 Unprocessable Content
4 ms
201 B
{ } JSON
Preview
Debug with AI
{
"detail": [
"Input should be greater than or equal to 1"
]
}

GET
http://127.0.0.1:8003/joke/query?friend=Jack&jokes_number=11
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
}
Body
422 Unprocessable Content
5 ms
199 B
{ } JSON
Preview
Debug with AI
{
"detail": [
"Input should be less than or equal to 10"
]
}



12. Начата разработка простого REST-сервиса с использованием библиотеки wikipedia. Сервис предоставляет доступ к информации из Wikipedia через HTTP-запросы. Для реализации использован фреймворк FastAPI. Все эндпоинты возвращают ответы в формате JSON и валидируются с помощью схем Pydantic.

Реализован эндпоинт, принимающий название статьи Wikipedia в качестве path-параметра

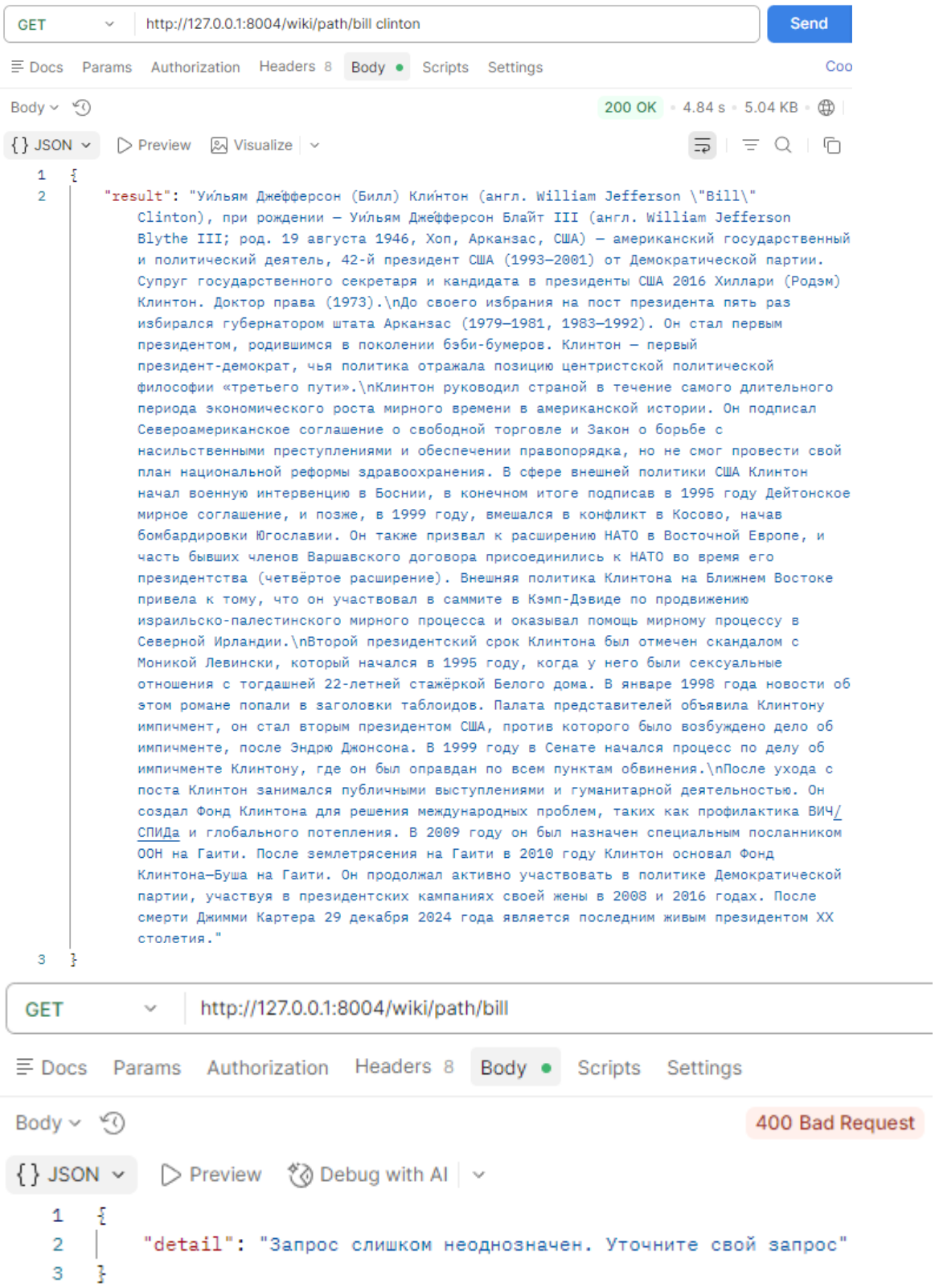
```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field, ValidationError
import wikipedia
from wikipedia import summary, DisambiguationError

wikipedia.set_lang("ru")

class SearchSummary(BaseModel):  2 usages
    result: str = Field(...)

app = FastAPI()

@app.get( path: "/wiki/path/{title}", response_model=SearchSummary)
def title_search(title: str):
    try:
        summary = wikipedia.summary(wikipedia.search(title)[0])
    except ValidationError as e:
        total_errors = []
        for error in e.errors():
            total_errors.append(error['msg'])
            raise HTTPException(status_code=422, detail=total_errors)
    except DisambiguationError as e:
        raise HTTPException(status_code=400, detail='Запрос слишком неоднозначен. Уточните свой запрос')
    except Exception as e:
        raise  HTTPException(status_code=500, detail=str(e))
    return SearchSummary(result=summary)
```



Эндпоинт:

- получает краткое описание статьи из Wikipedia
- возвращает результат в виде JSON

- использует схему ответа

13. Реализован эндпоинт, принимающий название статьи Wikipedia в качестве query-параметра

```python
@app.get(path: "/wiki/query", response_model=SearchSummary)
def title_search(title: str):
    try:
        summary = wikipedia.summary(wikipedia.search(title)[0])
    except ValidationError as e:
        total_errors = []
        for error in e.errors():
            total_errors.append(error['msg'])
            raise HTTPException(status_code=422, detail=total_errors)
    except DisambiguationError as e:
        raise HTTPException(status_code=400, detail='Запрос слишком неоднозначен. Уточните свой запрос')
    except Exception as e:
        raise  HTTPException(status_code=500, detail=str(e))
    return SearchSummary(result=summary)
```



Эндпоинт:

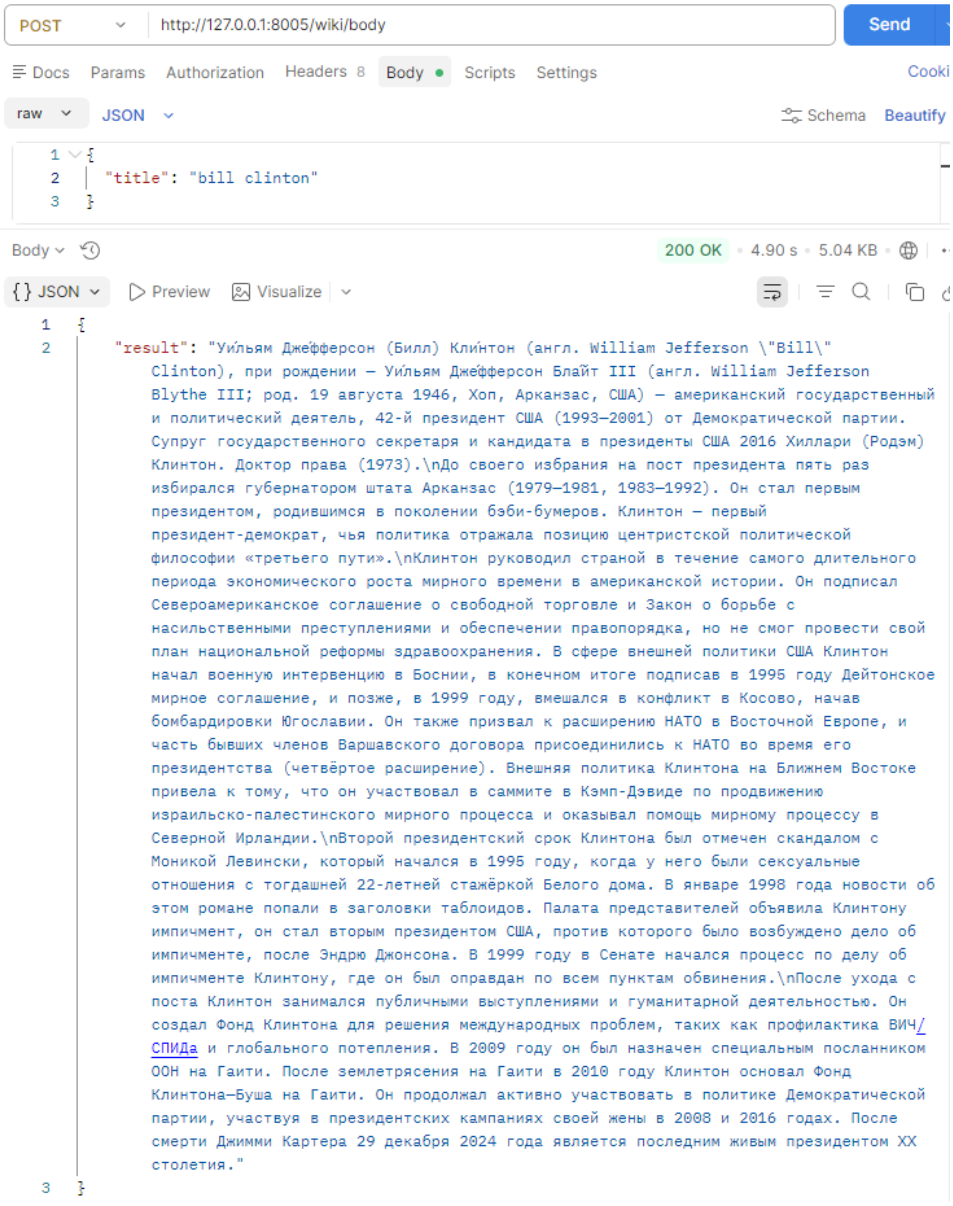
- выполняет тот же функционал, что и предыдущий
- принимает параметр через строку запроса
- возвращает результат по схеме

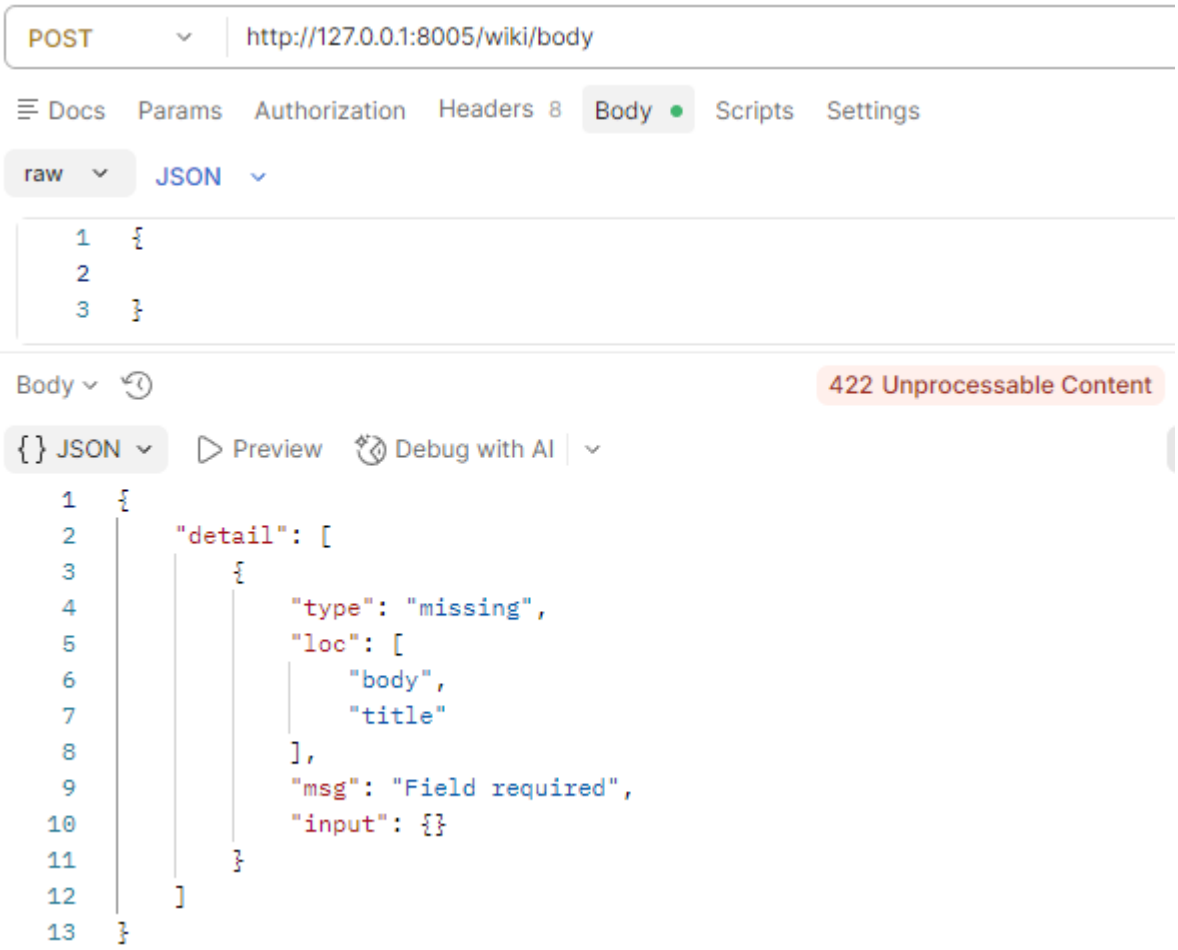
14. Реализован эндпоинт, принимающий название статьи Wikipedia через тело запроса (POST)

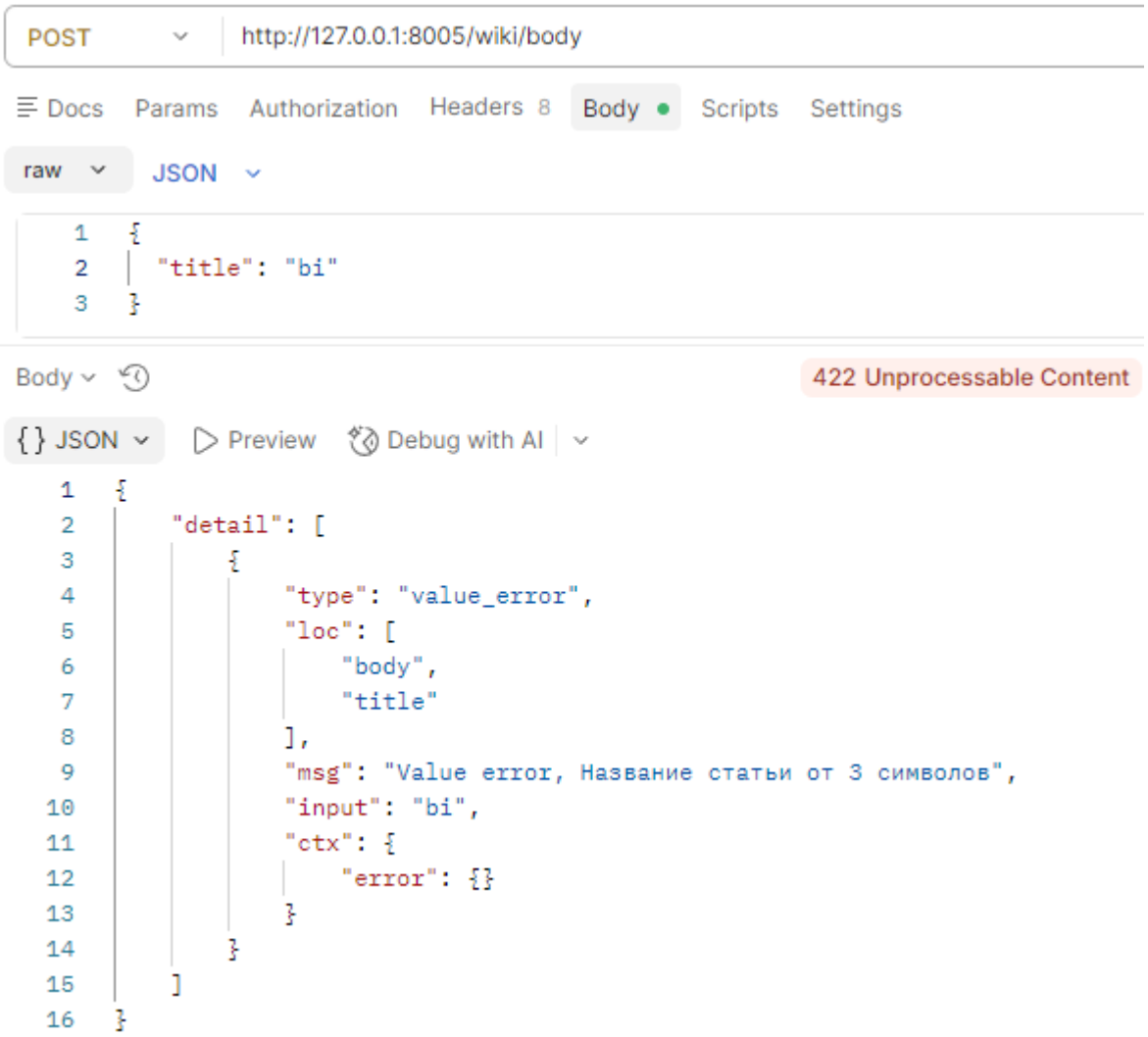
```python
@app.post( path: "/wiki/body", response_model=SearchSummary)
def title_search(title: SearchInput):
    try:
        summary = wikipedia.summary(wikipedia.search(title)[0])
    except ValidationError as e:
        total_errors = []
        for error in e.errors():
            total_errors.append(error['msg'])
            raise HTTPException(status_code=422, detail=total_errors)
    except DisambiguationError as e:
        raise HTTPException(status_code=400, detail='Запрос слишком неоднозначен. Уточните свой запрос')
    except MissingError as e:
        raise HTTPException(status_code=422, detail='Недостаточное тело запроса')
    except Exception as e:
        raise  HTTPException(status_code=500, detail=str(e))
    return SearchSummary(result=summary)
```

```python
class SearchInput(BaseModel):  1 usage
    title: str = Field(...)

    @field_validator('title')
    @classmethod
    def title_validation(cls, v):
        if len(v) < 3 :
            raise ValueError('Название статьи от 3 символов')
        return v.strip
```


POST
http://127.0.0.1:8005/wiki/body
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cooki
raw
JSON
Schema
Beautify
{
"title": "bill clinton"
}
Body
200 OK
4.90 s
5.04 KB
{} JSON
Preview
Visualize
{
"result": "Уи́льям Джеффе́рсон (Билл) Кли́нтон (англ. William Jefferson \"Bill\" Clinton), при рождении — Уи́льям Джеффе́рсон Блайт III (англ. William Jefferson Blythe III; род. 19 августа 1946, Хоп, Арканзас, США) — американский государственный и политический деятель, 42-й президент США (1993—2001) от Демократической партии. Супруг государственного секретаря и кандидата в президенты США 2016 Хиллари (Родэм) Клинтон. Доктор права (1973).\nДо своего избрания на пост президента пять раз избирался губернатором штата Арканзас (1979—1981, 1983—1992). Он стал первым президентом, родившимся в поколении бэби-бумеров. Клинтон — первый президент-демократ, чья политика отражала позицию центристской политической философии «третьего пути».\nКлинтон руководил страной в течение самого длительного периода экономического роста мирного времени в американской истории. Он подписал Североамериканское соглашение о свободной торговле и Закон о борьбе с насильственными преступлениями и обеспечении правопорядка, но не смог провести свой план национальной реформы здравоохранения. В сфере внешней политики США Клинтон начал военную интервенцию в Боснии, в конечном итоге подписав в 1995 году Дейтонское мирное соглашение, и позже, в 1999 году, вмешался в конфликт в Косово, начав бомбардировки Югославии. Он также призвал к расширению НАТО в Восточной Европе, и часть бывших членов Варшавского договора присоединились к НАТО во время его президентства (четвёртое расширение). Внешняя политика Клинтона на Ближнем Востоке привела к тому, что он участвовал в саммите в Кэмп-Дэвиде по продвижению израильско-палестинского мирного процесса и оказывал помощь мирному процессу в Северной Ирландии.\nВторой президентский срок Клинтона был отмечен скандалом с Моникой Левински, который начался в 1995 году, когда у него были сексуальные отношения с тогдашней 22-летней стажёркой Белого дома. В январе 1998 года новости об этом романе попали в заголовки таблоидов. Палата представителей объявила Клинтону импичмент, он стал вторым президентом США, против которого было возбуждено дело об импичменте, после Эндрю Джонсона. В 1999 году в Сенате начался процесс по делу об импичменте Клинтону, где он был оправдан по всем пунктам обвинения.\nПосле ухода с поста Клинтон занимался публичными выступлениями и гуманитарной деятельностью. Он создал Фонд Клинтона для решения международных проблем, таких как профилактика ВИЧ/СПИДа и глобального потепления. В 2009 году он был назначен специальным посланником ООН на Гаити. После землетрясения на Гаити в 2010 году Клинтон основал Фонд Клинтона—Буша на Гаити. Он продолжал активно участвовать в политике Демократической партии, участвуя в президентских кампаниях своей жены в 2008 и 2016 годах. После смерти Джимми Картера 29 декабря 2024 года является последним живым президентом XX столетия."
}


POST
http://127.0.0.1:8005/wiki/body
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
raw
JSON
1 {
2
3 }
Body
422 Unprocessable Content
{} JSON
Preview
Debug with AI
1 {
2 "detail": [
3 {
4 "type": "missing",
5 "loc": [
6 "body",
7 "title"
8],
9 "msg": "Field required",
10 "input": {}
11 }
12]
13 }



Эндпоинт:

- принимает данные через Pydantic-схему
- возвращает структурированный ответ
- валидирует входные данные

Вывод:

Я изучил разработку RESTful API, научился создавать свои api сервисы на fastapi и проводить валидацию данных при помощи pydantic.